

Développement d'un outil professionnel d'analyse granulométrique par vision artificielle.

Destiné au contrôle qualité industriel du ballast ferroviaire.

Présenté par

NGASSA NJIALE YANN LOIC

Etablissement académique

INSA HdF – Université polytechnique
de Valenciennes

Entreprise d'accueil

CIMES
180 rue Joseph-Louis-Lagrange 59300
Famars, France

Tuteur académique

M. Robert TOMCZAK

Tuteur entreprise

M. Stéphane VANHAESBROUCKE

Soutenance prévue le 08 septembre 2026

Remerciements

Je tiens à remercier sincèrement toutes les personnes qui ont contribué au bon déroulement de ce stage de fin d'études et à la réalisation des travaux présentés dans ce rapport.

J'adresse tout particulièrement mes remerciements à M. STEPHANE VANHAESBROUCKE, pour la confiance qu'il m'a accordée, sa disponibilité et la qualité de son encadrement. Ses interventions m'ont permis de mieux comprendre le besoin industriel, de questionner mes choix méthodologiques et d'orienter progressivement le développement de la solution afin qu'il réponde le mieux possible au monde industriel.

Je remercie BERRACHED TAREK pour l'encadrement qu'il m'a prodigué tout au long de ce stage, pour sa disponibilité, la richesse de nos échanges techniques et la confiance qu'il m'a témoigné dans la conduite autonome de ce projet.

Je remercie également AREZKI BOUKHALFA pour son aide précieuse sur les questions liées au développement de la solution. Ses remarques et la qualité de nos échanges ont enrichi mes réflexions et m'ont aidé à mieux comprendre plusieurs aspects du projet.

Je remercie l'ensemble des collaborateurs de CIMES pour leur accueil, leur bienveillance et le partage de leur expérience. Chacun, par sa disponibilité et ses encouragements a contribué à faire de ce stage une expérience particulièrement enrichissante, tant sur le plan professionnel que sur le plan humain.

Enfin, je salue la communauté scientifique et les contributeurs open-source des bibliothèques techniques, dont les outils ont constitué les fondations techniques de ce travail.

Résumé

Ce rapport présente les travaux réalisés au sein de CIMES dans le cadre du développement d'un outil professionnel d'analyse granulométrique par vision artificielle destiné au contrôle qualité industriel du ballast ferroviaire. L'objectif était de concevoir une application Python complète, déployable hors-ligne, capable d'estimer en temps réel la distribution dimensionnelle des granulats à partir d'un flux vidéo RTSP.

Le système repose sur un pipeline de segmentation d'instances par réseau de neurones convolutif YOLO, complété par une chaîne de calibration caméra rigoureuse. Une architecture modulaire, un système de licence cryptographique RSA-PSS hors-ligne et un packaging par PyInstaller ont permis de répondre aux contraintes de déploiement industriel. Les résultats obtenus démontrent la faisabilité d'une métrologie optique corrélée aux standards normatifs de tamisage NF EN 13450, tout en ouvrant des perspectives d'amélioration sur la généralisation du modèle de correction et l'intégration de la stéréovision.

Mots-clés : vision par ordinateur, granulométrie, YOLO-OBB, Python, ballast ferroviaire, cryptographie asymétrique, déploiement industriel.

Abstract

This report presents the work carried out at CIMES in the development of a professional computer vision tool for the industrial quality control of railway ballast. The objective was to design a complete, offline-deployable Python application capable of estimating the dimensional distribution of aggregates in real time from an RTSP video stream.

The system relies on an instance segmentation pipeline using a YOLO convolutional neural network, combined with a rigorous camera calibration. A modular architecture, an offline RSA-PSS cryptographic licensing system, and PyInstaller packaging addressed industrial deployment constraints. The results demonstrate the feasibility of optical metrology correlated with NF EN 13450 sieving standards, while opening perspectives for improving the generalization of the correction model and integrating stereovision.

Keywords: computer vision, particle size analysis, YOLO-OBB, Python, railway ballast, asymmetric cryptography, industrial deployment.

Table des matières

Remerciements	i
Résumé.....	ii
Abstract	ii
Liste des figures	v
Liste des tableaux	vi
Glossaires.....	vii
Introduction générale	1
Chapitre 1: Contexte et présentation de l'entreprise	2
1.1 Identification de la structure	2
1.2 Domaines d'activités	3
1.3 Objectif du stage	4
Chapitre 2 : Problématique et cadre du projet	5
2.1 La problématique industrielle	5
2.2 Contexte scientifique et technique.....	6
2.3 Objectifs et hypothèses	9
2.3.1 Objectifs.....	9
2.3.2 Hypothèses de travail.....	9
Chapitre 3: Méthodologie et outils	10
3.1 Approche et outils de développement	10
3.2 Technologies utilisées	10
3.3 Organisation et planification.....	11
Chapitre 4: Réalisations et résultats	13
4.1 Refactorisation architecturale et améliorations	13
4.1.1 Développement du sous-module App-calibration.....	13
4.1.2 Sous-module Reglage-parametre	13
4.1.3 Application Cimes-Yolo : refactorisation architecturale	14
4.2 Evolution du modèle de segmentation	16
4.2.1 Cause du changement de modèle	16
4.2.2 Etudes des alternatives et sélection de YOLO-OB.....	16
4.3 Implémentation du système de licence.....	21
4.4 Intégration des sous-modules.....	23
4.5 Packaging de l'application et déploiement	23
4.5.1 Choix de l'outil de packaging.....	24
4.5.2 Fichier de spécification	24

4.5.3 Packaging de l'outil autonome d'identification machine	24
4.5.4 Structure de livraison et procédure d'installation.....	24
4.6 Résultats obtenus	24
4.6.1 Ergonomie et expérience utilisateur.....	25
4.6.2 Stratégie de tests	25
Chapitre 5: Analyse critique et discussion	26
5.1 Bilan des forces et des choix validés.....	26
5.2 Limites et difficultés rencontrées	26
5.2.1 Limites méthodologiques.....	26
5.2.2 Limites techniques	27
5.2.3 Difficultés rencontrées.....	27
5.3 Discussion sur la chaine de mesure	28
Chapitre 6: Perspectives et bilan de l'expérience	29
6.1 Perspectives techniques et fonctionnelles	29
6.2 Bilan personnel	29
Conclusion	31
Bibliographie.....	32
Annexes	33
Annexes A : Code source partiel du projet.....	33
Annexes B : Images des différentes interfaces	37

Liste des figures

Figure 1: Logo CIMES	2
Figure 2: Plan de localisation	2
Figure 3 Représentation d'un ballast sur une voie ferrée.....	5
Figure 4: Analyse granulométrique par tamisage	5
Figure 5: lot de cailloux	6
Figure 6: yolo obb	7
Figure 7: Sous module app-calibration avant et après développement	13
Figure 8: Sous-module réglage-parametre avant et après développement	14
Figure 9: Arborescence du projet CIMES	15
Figure 10: Dataset annoté avec l'outil Roboflow	17
Figure 11: conversion des coordonnées des boites vers des boites orientés	18
Figure 12: différence entre les types boites.....	18
Figure 13: Courbe d'évolution de l'entrainement du modèle YOLO-OBB.....	20
Figure 14: Comparaison du temps d'inférence de Cellpose et de YOLO-OBB.....	20
Figure 15 système de gestion des licences	22
Figure 16 rendu final de l'interface principale	25
Figure 17: modèle d'image de lot de cailloux	29

Liste des tableaux

Tableau 1: Technologies mobilisées selon les besoins fonctionnels	10
Tableau 2: Planification indicatif du projet	12
Tableau 3: Protocole de comparaison des modèles de segmentation	17
Tableau 4: Tableau comparatif de l'entrainement des différents modèles.....	19
Tableau 5: Exigences fonctionnelles et contraintes du système de licence.....	21

Glossaires

Termes et acronymes

Terme	Définition
AEC	Architecture, Ingénierie et Construction, département de CIMES.
APE	Activité principale exercée, code caractérisant l'activité économique principale d'une entreprise.
BGR	Blue-Green-Red, format de couleur utilisé par la librairie OpenCV.
CUDA	Compute Unified Device Architecture, langage de programmation développé par NVIDIA.
CIR	Crédit Impôt Recherche.
JSON	JavaScript Object Notation.
MSE	Mean Squared Error, Métrique d'évaluation d'un modèle d'IA.
OBB	Oriented Bounding Box, boîte englobante orientée.
PKCS	Public-Key Cryptography Standards.
PSS	Probabilistic Signature Scheme, algorithme de cryptographie.
RSA	Rivest-Shamir-Adleman, algorithme cryptographique.
RTSP	Real Time Streaming Protocol.
SAM	Segment Anything Model, modèle de segmentation de Meta AI.
SHA	Secure Hash Algorithm.
SMTP	Simple Mail Transfer Protocol.
UUID	Universally Unique Identifier.

Introduction générale

Le contrôle granulométrique du ballast ferroviaire constitue une opération critique pour la sécurité et la maintenance des voies. La distribution de taille des cailloux conditionne directement la stabilité de la voie, sa capacité de drainage et sa résistance aux efforts dynamiques générés par le passage des trains. Un ballast trop fin ou trop hétérogène compromet la portance des traverses et accélère la dégradation de la voie, impliquant des campagnes de renouvellement coûteuses et des risques opérationnels. Les méthodes conventionnelles d'analyse fondées sur le tamisage manuel selon la norme NF EN 13450 fournissent des résultats fiables mais exigent un arrêt du chantier, une manipulation laborieuse des échantillons et un délai incompatible avec les contraintes de production industrielle.

La vision par ordinateur offre une alternative non destructive et instantanée. Elle repose sur l'acquisition d'images numériques du matériau et sur l'extraction automatique des propriétés morphologiques de chaque caillou à partir desquelles une courbe granulométrique peut être reconstituée. Cette approche se heurte néanmoins à des défis spécifiques au contexte industriel. Une analyse robuste des contours individuels est donc indispensable et ne peut être obtenue par les méthodes classiques de traitement d'image dès lors que les scènes présentent une forte complexité texturale.

Les modèles de détection par apprentissage profond, en particulier les architectures à boîtes orientées ouvrent de nouvelles perspectives pour ce problème. Ils permettent de localiser et de délimiter chaque particule indépendamment de son orientation et de son degré de recouvrement, là où un détecteur à boîtes alignées produirait des fusions indésirables. La chaîne de traitement ne se réduit cependant pas à l'inférence du modèle, elle comprend la calibration géométrique de la caméra, la correction empirique des biais systématiques entre mesure optique et tamisage physique, la visualisation dynamique des résultats et la production de rapports exploitables par les équipes terrain. Ces briques doivent de surcroît fonctionner dans un environnement industriel sans connexion internet ce qui impose des contraintes fortes en matière de déploiement et de protection logicielle. Le stage réalisé chez CIMES répond à l'ensemble de ces besoins. La mission consiste à concevoir et développer une application Python complète capable d'acquérir un flux vidéo RTSP depuis une caméra IP, de segmenter les particules par un modèle YOLO-OBB entraîné sur données réelles, d'extraire leurs propriétés morphologiques, d'appliquer une correction pour aligner les résultats sur les standards normatifs, et de restituer une courbe granulométrique interactive assortie d'un rapport PDF professionnel.

Le rapport présente successivement l'environnement du stage, la problématique industrielle, la démarche scientifique et technique, les résultats obtenus et leurs limites, puis les compétences développées au cours de stage.

Chapitre 1: Contexte et présentation de l'entreprise

Ce chapitre présente l'entreprise juridiquement porteuse du stage, son positionnement et l'organisation fonctionnelle dans laquelle la mission a été conduite.

1.1 Identification de la structure

Mon stage de fin d'études s'est déroulé au sein de l'entreprise CIMES pour Conception intégrée en mécanique et en structure, qui est un bureau d'études d'ingénierie fondé en 1989 à Valenciennes. CIMES est une société à responsabilité limitée (SARL), immatriculée sous le numéro SIREN 950 515 957 et dont l'activité principale déclarée relève de l'ingénierie et des études techniques (code APE 71.12B).

CIMES a été créée par cinq enseignants-chercheurs qui ont vu dans la simulation numérique une solution innovante pour appréhender le dimensionnement des structures et déployer de nouvelles stratégies de conception. Ils créent alors CIMES avec pour objectif d'apporter un regard spécialisé auprès des industriels en s'appuyant notamment sur les méthodes de calculs par éléments finis, et l'entreprise s'est depuis imposée comme un acteur reconnu de l'ingénierie de simulation multi-physique en France, avec cinq départements (Structure, Thermo-fluidique, Digital Lab., AEC, Logiciels) couvrant simulation numérique, ingénierie d'essais et traitement de données, depuis lors les travaux menés ont permis de confirmer la politique d'innovation de l'entreprise et d'être sélectionné lauréat du programme territoires d'industrie, les objectifs technologiques étant de développer des méthodes de co-simulation organisée grâce à la communication des différents codes de calculs et des différentes échelles physiques.. Le périmètre utile pour comprendre le stage est celui de la société opérationnelle française.



Figure 1: Logo CIMES

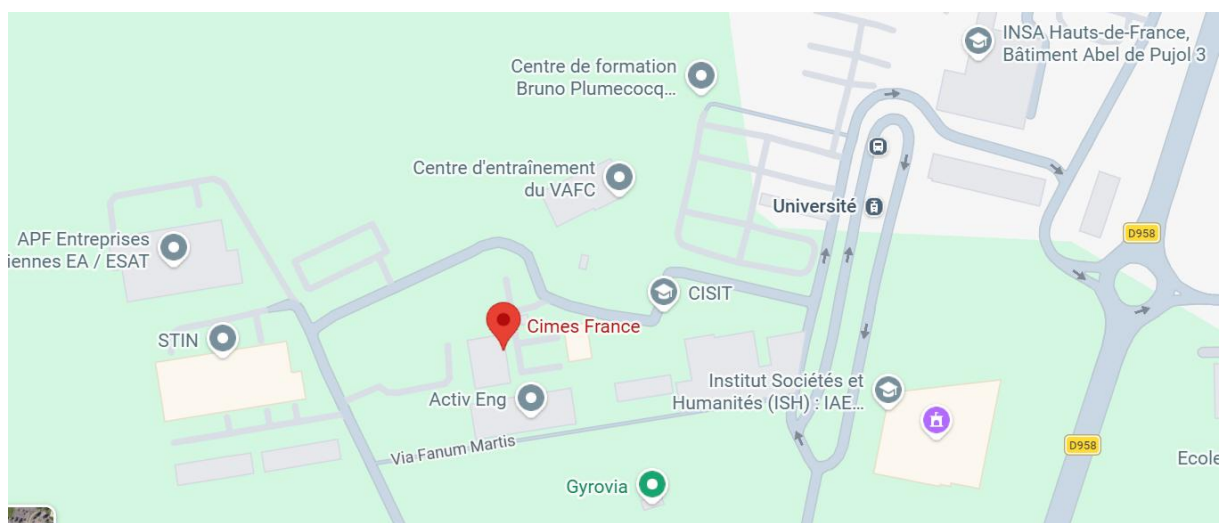


Figure 2: Plan de localisation

En termes de taille, l'entreprise compte en son sein 27 collaborateurs selon les données fournies par la responsable des ressources humaines. Son siège social se trouve à Valenciennes, avec une antenne secondaire à Lyon. CIMES intervient principalement auprès de grands acteurs industriels dans les secteurs automobile, ferroviaire, aéronautique et défense notamment. L'entreprise bénéficie par ailleurs de l'accréditation CIR (Crédit d'Impôt Recherche), témoignant de son engagement dans des démarches de recherche, développement et innovation, et a été distinguée lauréate du programme national « Territoires d'industrie ».

CAP-TRACK constitue une émanation plus récente de CIMES, dédiée à une problématique spécifique du secteur des transports et de la mobilité : le captage électrique, c'est-à-dire l'interaction entre un système de captage notamment le pantographe et une ligne d'alimentation électrique, typique des applications ferroviaires. CAP-TRACK est une plateforme d'essai ouverte à toutes les industries. Elle a pour activité d'accompagner les entreprises dans le développement de leurs produits et procédés liés au captage électrique, de tester et caractériser ces produits, et plus largement de conduire des activités de recherche et développement dans ce domaine. La structure a pour vocation de concevoir, construire et exploiter une plateforme technologique dédiée au captage de courant, en particulier dans le secteur des mobilités, afin d'apporter aux entreprises un support à leurs besoins d'innovation, d'homologation et d'expertise, tout en collaborant avec des universités et laboratoires de recherche pour intégrer les approches scientifiques les plus innovantes.



Figure 3: logo cap-track

Cette orientation s'inscrit dans la continuité des compétences historiques de CIMES en simulation numérique et ingénierie d'essais, que l'on retrouve par exemple dans ses travaux de certification virtuelle de l'interaction caténaire-pantographe pour le secteur ferroviaire. J'ai réalisé mon stage, au sein de l'équipe de CAP-TRACK en charge du développement d'un outil de granulométrie par vision artificielle. Le stage s'est déroulé dans les locaux du Technopole Transalley, un environnement mêlant startups, PME et structures de recherche, j'ai été intégré à une équipe pluridisciplinaire, ce qui m'a permis d'appréhender le projet qui m'a été confié non seulement sous l'angle purement informatique mais aussi sous l'angle des contraintes métiers propres au secteur de l'industrie.

1.2 Domaines d'activités

Le site institutionnel présente cinq départements : Structure, Thermo-fluidique, Digital Lab., AEC et Logiciels. Cette organisation associe plusieurs compétences complémentaires :

- le calcul de structures par éléments finis, notamment en statique, dynamique, fatigue, crash, optimisation et interaction fluide-structure ;
- les calculs thermiques et fluidiques ainsi que les problèmes multiphysiques ;
- la corrélation calcul-essai, la caractérisation des matériaux et l'instrumentation ;
- l'automatisation des chaînes de calcul, le traitement de données, la co-simulation et le développement d'outils logiciels.

1.3 Objectif du stage

L'objectif de ce stage était donc de participer au développement d'un outil professionnel d'analyse granulométrique par vision artificielle destiné à des clients industriels souhaitant automatiser le contrôle de la distribution dimensionnelle de ballast. Le logiciel doit permettre d'analyser en temps réel ou de manière différée la distribution de la taille des cailloux dans un lot à partir d'un flux de vidéo ou d'images enregistrées.

L'enjeu pour l'entreprise est double : d'une part un enjeu produit, soit livrer un logiciel robuste, packagé et deployable en environnement industriel et d'autre part il s'agit d'un enjeu commercial et de protection de la propriété intellectuelle car le produit doit s'accompagner d'un mécanisme de licence empêchant toute duplication ou usage non autorisé.

Mes missions ont donc couvert les axes suivants :

- Refactoriser le projet pour assurer et faciliter son évolution et sa maintenance dans le futur
- Implémentation des principes du clean code et versionning du projet avec git.
- Intégration des sous modules au sein de l'application principale.
- Participer à la conception, au développement et à l'amélioration du pipeline de segmentation et d'analyse d'images par apprentissage profond ;
- Concevoir et implémenter un système de licence logicielle fondé sur la cryptographie asymétrique RSA, avec empreinte matérielle des postes clients ;
- Documenter les développements réalisés : guide d'installation, documentation technique du logiciel etc.

Chapitre 2 : Problématique et cadre du projet

Ce chapitre présente la problématique de ce stage les contextes scientifiques et techniques liés à cette problématique.

2.1 La problématique industrielle

Le ballast désigne la couche de pierre concassées répandu sur le sol, sous la voie ferrée. Il sert à transmettre au sol les efforts engendrés par le passage des trains, sans se déformer par tassement. Il sert aussi à enchâsser les traverses afin d'assurer leur résistance aux déformations longitudinales, particulièrement importante pour la technique des longs rails soudés et assurer le drainage des eaux pluviales pour préserver la stabilité de la plateforme ferroviaire.

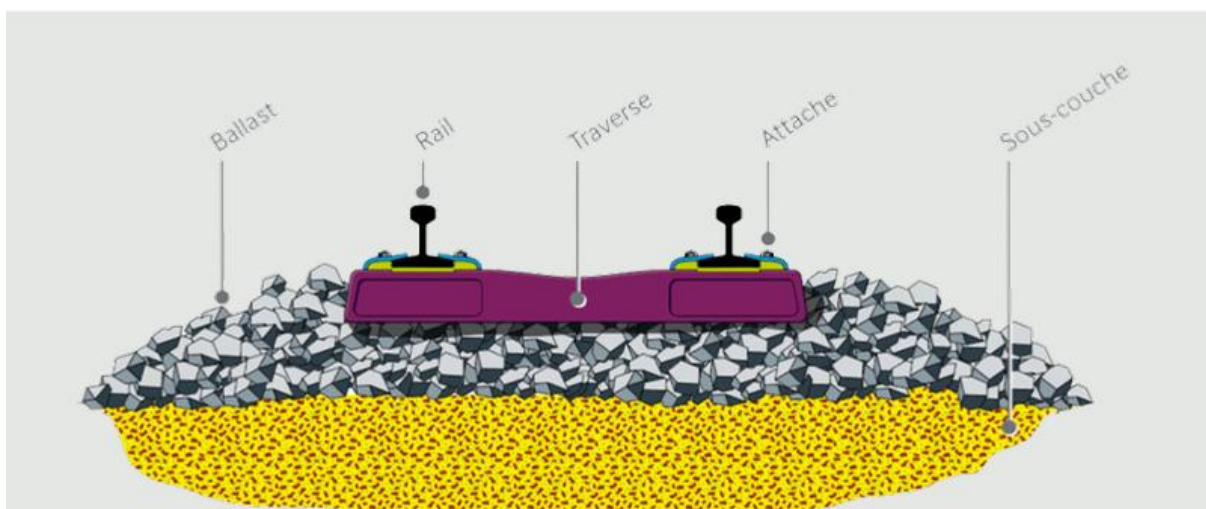


Figure 3 Représentation d'un ballast sur une voie ferrée

L'analyse granulométrique consiste à déterminer la distribution des tailles de cailloux constituant un ballast. La méthode de référence normalisée reste le tamisage mécanique, l'échantillon de cailloux est passé au travers d'une série de tamis de mailles décroissantes et la masse retenue à chaque niveau permet de construire une courbe granulométrique cumulée.

Cette méthode, bien qu'éprouvée et normalisée (NF EN & 13450) présente plusieurs limites opérationnelles majeures : elle nécessite un prélèvement d'échantillon physique donc un contrôle en différé non représentatif d'un flux continu, elle est chronophage soit plusieurs dizaines de minutes par mesure, elle immobilise du matériel et du personnel, peut être sujet à des erreurs humaines et elle ne permet pas un contrôle qualité en temps réel sur une chaîne de production industrielle. Impossible d'assurer un contrôle qualité systématique sur une chaîne de production industrielle.



Figure 4: Analyse granulométrique par tamisage

La problématique technique et industrielle à laquelle répond ce stage peut donc se formuler ainsi : Comment concevoir un système de vision par ordinateur capable d'estimer, à partir d'un flux vidéo, une courbe granulométrique fiable et corrélée aux mesures de tamisage physique de référence ?

Elle se décompose en plusieurs sous-problèmes techniques traités durant le stage : la détection et la segmentation des particules dans une image souvent bruitée et présentant des recouvrements entre cailloux, l'établissement d'une correspondance métrique fiable entre l'espace image en pixels et l'espace physique en millimètres et enfin la sécurisation de la distribution commerciale du logiciel.

2.2 Contexte scientifique et technique

La segmentation d'instances consiste à détecter et délimiter individuellement chaque objet présent dans une image, par opposition à la segmentation sémantique qui se contente de classer chaque pixel par catégorie sans distinguer les occurrences. Dans le contexte de la granulométrie par vision, chaque caillou doit être identifié et isolé de ses voisins, y compris lorsqu'ils se chevauchent partiellement, ce qui est une configuration fréquente sur un tapis de convoyage de granulats.



Figure 5: lot de cailloux

Plusieurs familles d'approches existent pour ce type de tâche: les méthodes classiques de traitement d'image comme le seuillage, l'algorithme watershed ou encore des opérateurs morphologiques. Les réseaux de segmentation par masque comme les R-CNN, Cellpose. Les détecteurs modernes à une seule passe tels que la famille YOLO. L'émergence de la vision par ordinateur a permis de déporter cette analyse vers des méthodes non intrusives et temps réel. La photogrammétrie de particules repose sur l'acquisition d'images suivie d'une segmentation permettant d'extraire les contours de chaque grain. Contrairement aux approches classiques de traitement d'images qui peinent face aux problèmes de superpositions et aux textures hétérogènes, qui sont des phénomènes courants dans les matériaux comme le ballast ferroviaire, les méthodes par apprentissage profond ont démontré une capacité de généralisation supérieure.

Le projet mobilise spécifiquement un modèle d'apprentissage profond de boîtes orientées dite YOLO-OBb qui prédit pour chaque objet un rectangle englobant orienté défini par 04 sommets

plutôt qu'un rectangle aligné sur les axes de l'image. Cette approche est particulièrement adaptée à la détection de particules allongées comme c'est le cas pour les cailloux de forme irrégulière observés sous des angles variables.

Sur le plan architectural, les détecteurs de la famille YOLO reposent sur un réseau de neurones convolutif unique, découpant l'image en une grille et prédisant simultanément, pour chaque cellule de cette grille, la probabilité de présence d'un objet, ses coordonnées géométriques et sa classe. Cette conception en une seule passe explique la rapidité d'inférence caractéristique de YOLO, qui est une condition nécessaire à un traitement quasi-temps réel sur un flux de vidéo industriel continu. La variante des boîtes orientées de ce modèle, ajoute aux quatre coordonnées d'une boîte alignée un paramètre d'angle supplémentaire permettant au réseau d'épouser plus fidèlement le contour allongé et non orthogonal d'un caillou. Tout en prédisant des boîtes englobantes orientées paramétrées par le centre (x, y) , la largeur w , la hauteur h et l'angle de rotation θ , cette architecture minimise l'espace vide autour des particules et améliore la précision de l'estimation des dimensions réelles.

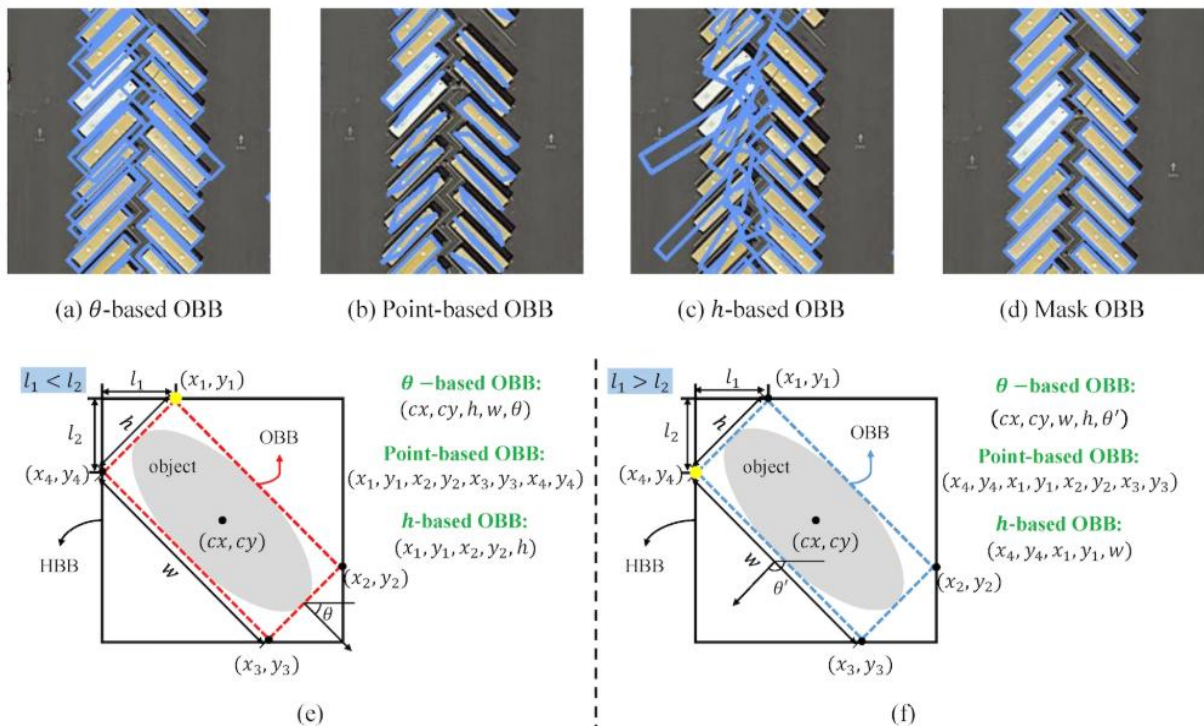


Figure 6: yolo obb

Dans le cadre du projet CIMES, l'inférence est réalisée via le Framework Ultralytics, qui fournit une implémentation optimisée de YOLO-OBB avec support GPU CUDA. La segmentation produite permet d'extraire, pour chaque particule détectée, des descripteurs morphologiques (axe majeur a et mineur b) via la bibliothèque *Scikit-image*, fondée sur la morphologie mathématique et l'analyse de formes binaires.

Mais avant d'effectuer toute opération de segmentation sur les images fournies par la caméra, 02 corrections majeures s'imposent : Toute mesure dimensionnelle obtenue par caméra nécessite une conversion rigoureuse entre le repère image en pixels et le repère physique en millimètres. L'objectif final de la granulométrie industrielle est de produire une distribution de tailles en millimètres, directement comparable aux normes de tamisage. Il devient donc indispensable de modéliser la chaîne de projection optique et d'établir la correspondance

métrique entre le plan image et le plan objet, ce qui constitue l'étape de calibration caméra. Elle nécessite la modélisation géométrique de la caméra et la correction des distorsions optiques. Cette chaîne est assurée par deux opérations : La distorsion optique introduite par l'objectif corrigée classiquement par le modèle de calibration de caméra de Zhang, implémenté dans OpenCV et la perspective de prise de vue, corrigée par une transformation d'homographie qui ramène le plan observé à une vue orthogonale (vue de dessus) lorsque la caméra n'est pas parfaitement perpendiculaire à la surface mesurée.

La distribution commerciale d'un logiciel destiné à des clients industriels pose la question de la protection contre la copie et l'usage non autorisé. Les schémas de licence hors-ligne reposent classiquement sur la cryptographie asymétrique RSA : une clé privée, détenue uniquement par l'éditeur, signe numériquement un fichier de licence contenant les droits accordés au client ; une clé publique embarquée dans l'application cliente permet de vérifier cette signature sans jamais pouvoir en produire une nouvelle. Ce mécanisme associé à une empreinte matérielle du poste client permet un contrôle d'activation fonctionnant intégralement hors-ligne contrainte impérative dans un contexte industriel où les postes ne disposent pas nécessairement d'un accès internet. Le principe du RSA repose sur la difficulté calculatoire de factoriser le produit de deux grands nombres premiers. Une paire de clés (publique, privée) est dérivée de ce produit ; les opérations de chiffrement/déchiffrement exploitent une propriété d'exponentiation modulaire réciproque entre les deux clés. Pour la signature de documents, le standard PKCS#1 recommande l'usage d'un schéma de padding probabiliste tel que PSS, combiné à une fonction de hachage cryptographique ici SHA-256, plutôt qu'une signature RSA appliquée directement sur les données brutes approche jugée obsolète et vulnérable à certaines attaques. C'est précisément ce schéma (RSA-2048 + PSS + SHA-256) qui a été retenu pour le système de licence CIMES, conformément aux recommandations actuelles en matière de sécurité cryptographique pour ce type d'application.

Le projet CIMES s'inscrit à l'intersection de 4 domaines : la vision par ordinateur industrielle, la métrologie optique, la science des matériaux granulaires et le déploiement. Les verrous techniques spécifiques relevés dans ce stage sont :

- La robustesse de la détection : garantir une segmentation fiable sur des flux vidéo RTSP en conditions industrielles.
- La traçabilité métrologique : établir une chaîne de mesure certifiable entre l'image numérique et la référence physique tamisée.
- L'ergonomie de calibration : concevoir des outils accessibles à des opérateurs non-spécialistes, tout en préservant la rigueur mathématique sous-jacente.
- Déploiement sécurisé : livrer un logiciel packagé, robuste et protégé contre la copie, fonctionnant hors-ligne

Ces éléments justifient l'architecture modulaire adoptée, la séparation des couches métiers et de la logique algorithmique, ainsi que le recours à une cryptographie asymétrique pour le contrôle de licence, répondant aux contraintes de déploiement hors-ligne propres aux environnements industriels.

2.3 Objectifs et hypothèses

2.3.1 Objectifs

L'objectif général du stage était de participer au développement d'un outil professionnel d'analyse granulométrique par vision artificielle, destiné à des clients industriels souhaitant automatiser le contrôle de la distribution dimensionnelle de ballast.

Plus spécifiquement le logiciel devait permettre :

- L'analyse en temps réel ou différée de la distribution granulométrique à partir d'un flux vidéo ou d'images enregistrées.
- La production d'une courbe granulométrique interactive et d'un rapport PDF professionnel.
- Le fonctionnement autonome hors-ligne dans un environnement industriel sans connectivité internet.
- La protection par un mécanisme de licence matérialisé.

L'enjeu pour l'entreprise était double : un enjeu produit, soit livrer un logiciel robuste packagé et un enjeu commercial soit accompagner la livraison d'un mécanisme de licence empêchant toute duplication ou usage non autorisé.

2.3.2 Hypothèses de travail

Le projet repose sur les hypothèses fondatrices suivantes :

H1 Corrélation 2D/3D : la distribution des dimensions apparentes mesurées en projection 2D sur le plan image est corrélée à la distribution granulométrique réelle (3D) obtenue par tamisage, moyennant un facteur de correction empirique calibré sur données réelles.

H2 Adéquation de YOLO-OB : l'architecture YOLO-OB est structurellement plus adaptée à la géométrie irrégulière des granulats allongés que les détecteurs à boîtes alignés ou les segments sémantiques généralistes, tout en maintenant une latence compatible avec le temps réel.

H3 Calibration stationnaire : la caméra étant fixée en position rigide au-dessus du tapis de convoyage, une calibration intrinsèque et extrinsèque effectuée lors de l'installation suffit à garantir la traçabilité métrique sur la durée d'exploitation, sous réserve de stabilité mécanique.

H4 Sécurité par asymétrie : le schéma RSA-2048/PSS/SHA-256, combiné à une empreinte matérielle du poste client, offre un niveau de sécurité suffisant pour un déploiement hors-ligne ciblant des utilisateurs non spécialisés en cryptanalyse

Chapitre 3: Méthodologie et outils

Ce chapitre présente la démarche de travail suivie tout au long du stage, les outils et technologies mobilisés, ainsi que l'organisation retenue pour mener à bien le développement de plusieurs briques logicielles interdépendantes dans un temps réduit.

3.1 Approche et outils de développement

Compte tenu de la nature exploratoire du projet et de l'articulation nécessaire avec des modules métiers encore en évolution, une démarche itérative et incrémentale a été privilégiée, proche des principes agiles sans mise en œuvre formelle d'un Framework de type scrum. Le travail a été organisé en cycles courts de développement, ponctués de points de synchronisation réguliers avec mon responsable, permettant d'ajuster les priorités en fonction de retours obtenus et des contraintes découvertes en cours de route comme celles liées au monde de l'industrie.

Le principe directeur qui a structuré l'ensemble de la démarche est la modularité. Plutôt que de développer directement la plupart des fonctionnalités directement au sein de l'application principale, celles-ci ont été conçues comme des sous modules autonomes et testables indépendamment avant d'être intégrées à l'application industrielle finale sous forme de sous-modules Git. Cette approche a permis de développer, déboguer et valider chaque brique fonctionnelle isolément, avec son propre cycle de test, avant intégration, réduisant ainsi la complexité de mise au point du système complet.

3.2 Technologies utilisées

L'ensemble de la chaîne logicielle a été développée en python, langage déjà largement utilisé dans l'écosystème de traitement d'image, d'apprentissage profond et permettant un développement rapide d'interfaces graphiques de bureau via Tkinter choix cohérent avec la cible de déploiement. Le tableau suivant synthétise les principales bibliothèques et outils mobilisés selon les besoins fonctionnels du projet.

Tableau 1: Technologies mobilisées selon les besoins fonctionnels

Besoin fonctionnel	Technologie
Gestion des versions	Git / GitHub
Gestion du projet et dépendances	uv
Segmentation	Ultralytics Yolo
Cryptographie asymétrique	RSA, PSS SHA-256
Interface graphique de bureau	Tkinter
Traitement d'image	OpenCv
Génération de rapport PDF	ReportLab
Visualisation des courbes	Matplotlib
Extraction de propriétés géométriques	Scikit-image
Qualité de code / linter	Pylint, Black

Le choix du gestionnaire de projet uv, venant en remplacement de 03 outils historique pip, venv et poetry a été retenu pour sa rapidité de résolution des dépendances et reproductibilité garantie par le fichier de verrouillage uv.lock, un point important compte tenu de la lourdeur des dépendances liées à l'apprentissage profond (pytorch et ultralytics) et de la nécessité de figer un environnement stable pour la génération de l'exécutable final.

3.3 Organisation et planification

Le développement s'est organisé chronologiquement autour de trois dépôts Git distincts, reflétant la logique modulaire décrite plus haut :

- **Reglage-parametre** : sous module de l'application de réglage des paramètres de correction granulométrique.
- **App-calibrage** : sous module contenant l'application d'inspection visuelle et de calibration métrique à partir d'images.
- **Cimes** : qui est l'application industrielle principale, intégrant la segmentation Yolo, la chaîne de calibration caméra complète, le calcul granulométrique, le système de licence ainsi que les deux modules précédents rattachés en tant que sous-modules Git.

Cette structuration en sous modules Git présente un double avantage méthodologique car elle permet de faire évoluer chaque outil indépendamment avec son propre historique de versions tout en garantissant que l'application principale référence toujours une révision précise et validée de chaque module, évitant les régressions liées à des modifications non maîtrisées. La coordination avec mon tuteur s'est faite au moyen de points d'avancement réguliers, de revues de code informelles et de tests fonctionnels menés conjointement sur les données réelles par l'entreprise.

La validation des développements a reposé sur plusieurs types de vérification complémentaires : des tests fonctionnels manuels sur chacune des applications autonomes avec une vérification du comportement de l'interface, de l'import et de l'export des données, de la cohérence des calculs. Une vérification de non régression lors de l'intégration des sous-modules dans l'application principale, une comparaison quantitative entre les courbes granulométriques obtenues par le logiciel et les courbes de référence obtenues par tamisage physique, utilisées pour piloter l'ajustement des paramètres de correction ainsi qu'une vérification systématique, via l'outil pylint et le formateur black de la qualité et de l'homogénéité du code produit, intégrée en partie sous forme d'intégration continue avec l'outil GitHub Actions.

L'ensemble du projet a été versionné sur des dépôts hébergés sur GitHub Cimes France. Chaque dépôt suit une convention de messages de commit typée, facilitant la lecture de l'historique et la traçabilité des évolutions fonctionnelles. Le dépôt **Reglage-parametre** embarque en particulier un workflow d'intégration continue GitHub actions déclenchant automatiquement une analyse pylint à chaque poussée de code, garantissant qu'aucune régression de qualité comme une erreur de style, imports inutilisés ou une complexité excessive n'est introduite silencieusement.

Sur le dépôt principal, plusieurs passes de nettoyage de code ont été menées en cours du stage, avec un objectif de maintenabilité à long terme du produit, celui-ci étant amené à être repris par d'autres membres de l'équipe et à évoluer après la fin du stage. La gestion des dépendances via uv et le fichier de verrouillage garantit par ailleurs la reproductibilité exacte de l'environnement de build, point sensible compte tenu du nombre et du poids des dépendances liées à

l'apprentissage profond. Le déroulement du stage peut être synthétisé de façon indicative selon les grandes phases suivantes :

Tableau 2: Planification indicatif du projet

Phase	Contenu principal	Livrable associé
1 Prise en main des	Découverte de l'environnement, du domaine métier, de la base de code existante et des outils	
Module de calibration	Refactorisation, amélioration et développement de fonctionnalités du module	Module fonctionnel et testé
Module de correction	Refactorisation, amélioration et développement de fonctionnalités du module	Module fonctionnel et testé
Application principale	Refactorisation, amélioration et développement de fonctionnalités du module	
Intégration des modules	Rattachement des 02 modules en sous-modules Git et résolution des conflits d'intégration	Application CIMES unifiée
Segmentation Yolo	Remplacement du système de segmentation présent par un pipeline de segmentation contenant yolo obb	Pipeline opérationnel
Système de licence	Conception et implémentation du système de licence RSA hors-ligne	Système de licence opérationnel
Packaging	Génération de l'exécutable de l'application	Exécutable
Phase de test	Test de l'application sur de véritable utilisateurs et correction de bugs	
Documentation	Rédaction de la documentation technique, guide utilisateur et guide d'installation de l'app	Documentation finale

Chapitre 4: Réalisations et résultats

Ce chapitre présente l'ensemble des travaux réalisés au cours du stage de la prise en main du code source du projet à la livraison du logiciel prêt à l'emploi et les résultats obtenus à chaque étape.

4.1 Refactorisation architecturale et améliorations

4.1.1 Développement du sous-module App-calibration

Ce sous module a connu une évolution marquée par 03 phases :

- **Phase de développement** : qui a été marquée par l'ajout de quelques fonctionnalités permettant d'améliorer le module tels que l'ajout de la molette de zoom pour permettre d'améliorer l'expérience des utilisateurs, la correction des imports dans le code et l'introduction d'une persistance des mesures, permettant de conserver l'état de calibration entre les sessions. La gestion des erreurs a été systématisée et l'adaptation aux contraintes du déploiement Py installer a été traitée. Enfin une amélioration de l'interface utilisateur a été effectuée.
- **Phase de structuration et refactorisation** : durant cette phase le code a été soumis à une analyse statique via les outils pylint et black pour une mise en conformité aux standards python. La logique de mesure a été extraite de l'interface pour former un module dédié, conformément au principe de séparation des responsabilités, un travail de documentation a été initié via un readme.
- **Phase d'intégration** : Pour finir ce module a été adapté pour fonctionner comme application autonome tout en restant compatible avec le lancement depuis l'application principale.

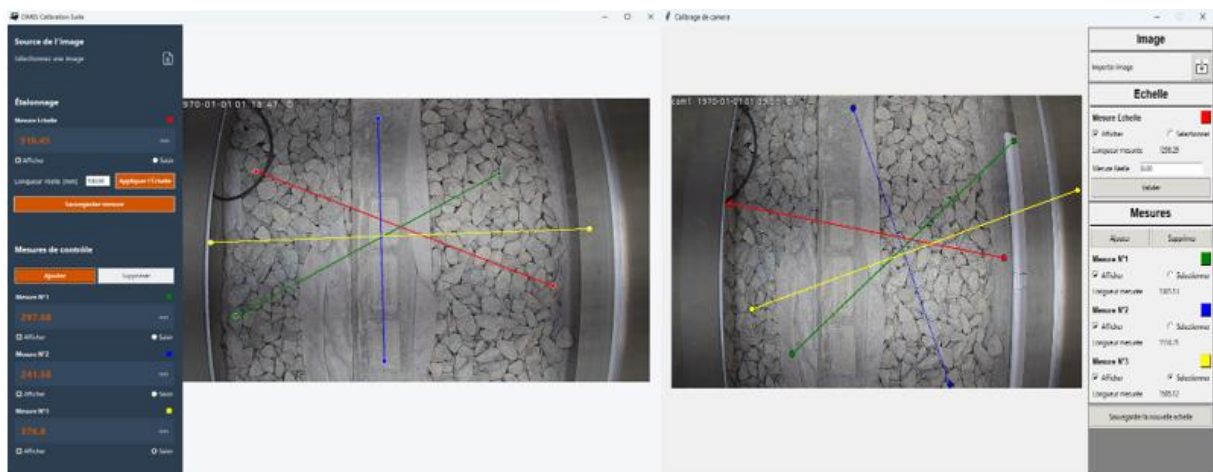


Figure 7: Sous module app-calibration avant et après développement

4.1.2 Sous-module Reglage-parametre

La structure du projet a été posée puis enrichie d'une interface structurée. La sécurité a été prise en compte dès les premières modifications par la suppression de secrets en clair.

Un travail conséquent de nettoyage a été mené ; mise à jour du linter sur plusieurs itérations, des ajouts de commentaires au code et une mise à jour des dépendances.

La gestion des erreurs a été renforcée via des blocs Try/excepté, garantissant que les exceptions liées à l'importation de données ou au calcul de l'optimisation ne fassent plus planter le module. Sur le plan algorithmique, la fonction d'erreur quadratique a été corrigée et le format d'affichage de l'offset a été standardisé. Le module a ensuite été adapté pour s'intégrer à l'architecture de l'application principale avec la synchronisation des ressources et des facteurs de conversion. La mise à jour finale des composants graphiques a permis d'unifier l'expérience utilisateur avec celle de l'application principale.

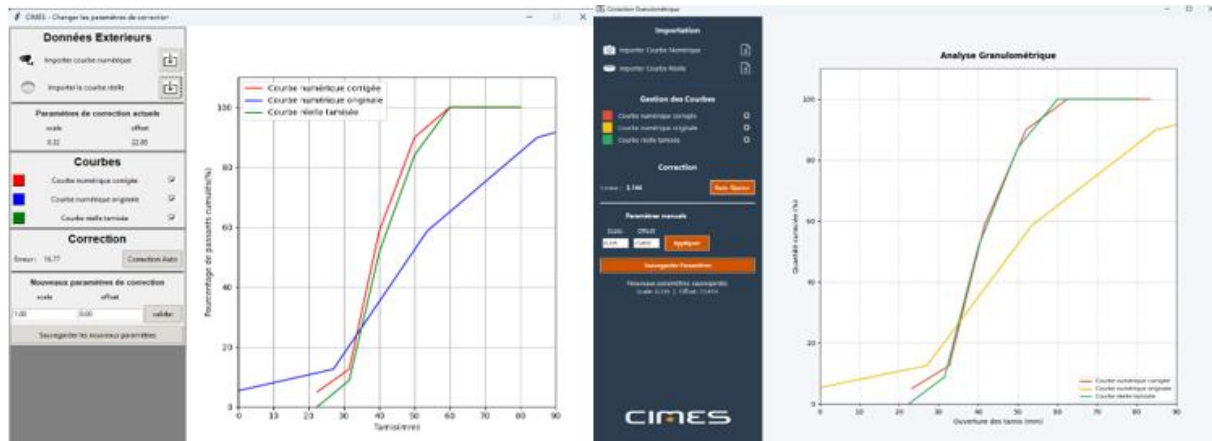


Figure 8: Sous-module réglage-parametre avant et après développement

4.1.3 Application Cimes-Yolo : refactorisation architecturale

Au début du stage l'application CIMES existait sous la forme d'un script monolithique unique, rassemblant l'acquisition vidéo, le traitement d'image, l'interface graphique et la génération de rapports dans quelques fichiers exécutés de manière linéaire. Cette approche, bien que fonctionnelle pour un prototype de preuve de concept, présentait plusieurs limitations critiques dans une perspective industrielle : absence de séparation des responsabilités, difficulté de maintenance, gestion d'erreur et capture d'exceptions.

La première étape de la réalisation a donc consisté en une refonte architecturale complète, guidée par les principes du clean code et de la programmation modulaire. Pour l'application principale et ses 02 sous-modules le code a été découpé en trois couches distinctes :

- Core : contenant la logique métier du code
- Ui : contenant tous les éléments liés l'interface utilisateur de l'app
- Utils : contenant les services transverses comme la configuration et la journalisation

Tout ceci conformément au pattern architectural Model-view et à une logique de séparation des préoccupations, ceci a donc permis d'isoler les modules critiques comme la segmentation Yolo en composants testables indépendamment, et de faciliter l'intégration des nouvelles fonctionnalités développées par la suite.

```

Cimes/
├─ main.py           # Point d'entrée de l'application principale
├─ pyproject.toml    # Configuration du projet et dépendances (uv)
├─ uv.lock           # Verrouillage des versions (généré par uv)
├─ .python-version   # Version Python imposée (3.13)
├─ best.pt           # Modèle YOLO-OBB entraîné (chargement unique en cache)
├─ assets/           # Logos, icônes et fichiers de calibration (.npz)
├─ keys/             # Clés de sécurité (public_key.pem pour vérification)
├─ src/
│   ├─ core/         # Moteurs de calcul (segmentation, stats, calibration)
│   ├─ license/       # Gestion de la licence (RSA, empreintes, dialogue Tkinter)
│   └─ ui/            # Interface graphique (Tkinter)
│       ├─ views/     # Vues principales (Mesure, Courbe, Paramètres, etc.)
│       ├─ widgets/   # Composants UI réutilisables et utilitaires
│       └─ app_init/  # Logique d'initialisation et variables d'état
├─ utils/            # Gestion des fichiers, config, mail SMTP et logs
├─ tools/            # Scripts internes (génération de clé, signature, machine_id)
└─ modules/          # Sous-applications et outils externes

```

Figure 9: Arborescence du projet CIMES

Un travail conséquent a ensuite été mené sur la gestion des exceptions, qui était auparavant absente. Dans le module de segmentation, des validations préalables systématiques ont été implémentées : vérification de la nullité de l'image, contrôle du format soit 3 canaux BGR, détection de la disponibilité des dépendances critiques comme ultralytics et Scikit-image via des blocs Try/except avec fallback explicite, et vérification de l'existence du modèle YOLO-OBB avant chargement. Chaque étape sensible tel que le chargement du modèle, inférence, analyse morphologique est désormais encapsulée dans une structure de gestion d'erreur qui logue l'état de l'opération et propage des exceptions typées avec messages explicites, permettant à la couche UI d'afficher des boîtes de dialogue informatives à l'utilisateur final plutôt que de laisser l'application planter silencieusement.

Pour la gestion de journalisation et de la traçabilité un système de logging structuré a remplacé les affichages console épars. Des préfixes sémantiques ont été standardisés ([OK], [ERREUR], [INFO], [DEBUG], [ATTENTION], [SAUVEGARDE], [TRANSMISSION], [EMAIL], [CALIBRATION], [LOG]) pour permettre un filtrage rapide des événements et faciliter le diagnostic en environnement de production. Dans main.py, une redirection conditionnelle des flux stdout et stderr vers un fichier de debug a été mise en place, activable uniquement via la variable d'environnement CIMES_DEBUG=1, garantissant ainsi qu'aucune donnée sensible n'est loguée en clair en production. Ce mécanisme s'avère particulièrement utile pour le débogage des déploiements PyInstaller, où les chemins d'accès et le chargement des DLLs CUDA nécessitent une traçabilité fine.

La gestion de configuration centralisée et migration de données, les paramètres de l'application étaient historiquement dispersés dans des fichiers texte non structurés (.txt) aux emplacements variés. Un système de configuration centralisée a été développé, s'appuyant sur un répertoire utilisateur standard et un format JSON structuré pour tous les paramètres (correction granulométrique, calibration caméra, configuration SMTP). Des fonctions de migration automatique ont été implémentées (load_correction_parameters, load_conversion_param) pour convertir silencieusement les anciens formats .txt vers le nouveau standard JSON, assurant la rétrocompatibilité lors des mises à jour chez les clients existants. La persistance des données

est désormais atomique et typée, avec gestion des erreurs et retour aux valeurs par défaut documentées en cas de corruption.

Le code a ensuite été rendu compatible avec la compilation en exécutable autonome via PyInstaller. Une abstraction des chemins de fichiers a été mise en place pour distinguer dynamiquement l'environnement de développement donc les fichiers sources de l'environnement déployé. Cette abstraction garantit que les ressources tels que les modèles YOLO, fichiers de calibration et icônes sont localisées correctement quel que soit le contexte d'exécution. Par ailleurs, un mécanisme de cache global évite le rechargement du modèle de deep learning à chaque capture, réduisant significativement la latence en traitement temps réel. La gestion des DLLs de PyTorch dans les exécutables compilés a également été adressée dans main.py par une résolution dynamique des chemins de bibliothèques CUDA.

L'ensemble du code a été documenté via des docstrings Google-style détaillées, spécifiant les types des paramètres, les valeurs par défaut, les exceptions levées et les structures de retour. Les modules utilitaires ont été conçus comme des APIs à usage unique, avec des signatures explicites et une gestion des cas limites de fichier inexistant, configuration SMTP incomplète ou PDF déjà ouvert. Dans le module de correction granulométrique, la logique mathématique a été isolée dans engine.py sous forme de fonctions pures sans effet de bord, tandis que la couche UI (correction_panel.py, graph.py) se concentre exclusivement sur la présentation et l'interaction, conformément au principe *Single Responsibility*. Enfin, une centralisation des styles graphiques et des composants réutilisables assure la cohérence visuelle et facilite la maintenance de l'interface sur l'ensemble des trois applications.

4.2 Evolution du modèle de segmentation

4.2.1 Cause du changement de modèle

La première approche de segmentation retenue pour ce projet avant mon arrivée reposait sur Cellpose un réseau de segmentation initialement conçu pour la biologie et la détection de cellule, capable de produire des masques de segmentation précis sans nécessiter un réentraînement complet pour de nouveaux types d'objets. Ce choix initial s'expliquait par la disponibilité d'un modèle généraliste performant, ne nécessitant pas de jeu de données annoté volumineux pour obtenir des premiers résultats exploitables.

Cependant, les tests menés en conditions réelles ont mis en évidence des limites pour l'usage industriel visé, en effet le temps d'inférence de Cellpose mesuré entre 5 et 10 secondes par image sur des images cibles et la taille du modèle qui embarquait également le modèle SAM de Facebook le rendait incompatible avec l'exigence d'un contrôle quasi-temps réel sur un flux vidéo continu. Une telle latence à l'échelle d'un poste de contrôle qualité destinée à traiter des captures répétées tout au long d'une journée de production aurait entraîné soit une mesure de cadence très réduite, soit une file d'attente croissante de traitement en cas de captures rapprochées, étant donné que le modèle est embarqué dans l'application cela aurait fait exploser la taille de cette dernière. Des situations incompatibles avec l'objectifs de supervision continue du produit, ces contraintes ont constitué les causes de la remise en question du choix initial et de l'étude d'architectures alternatives plus rapides.

4.2.2 Etudes des alternatives et sélection de YOLO-OBB

Face à ce constat plusieurs architectures de segmentation ont été comparées. Afin d'objectiver la décision de changement de modèle. Une étude comparative a été menée entre Cellpose, l'algorithme de Watershed, Mask R-CNN, Yolo 26m et Yolo OBB 26m. Selon un protocole

résumé dans le tableau. L'objectif était d'isoler la variable « architecture de segmentation » en fixant l'ensemble des autres conditions expérimentales.

Tableau 3: Protocole de comparaison des modèles de segmentation

Paramètre du protocole	Valeur retenue
Matériel de test	Poste équipé d'une carte graphique dédiée
Jeu d'images de test	Sous-ensemble d'image de lots de cailloux représentatif, identique pour les différents modèles
Métrique principale mesurée	Temps d'inférence par image (ms) F1 score (précision et recall) Qualité de la segmentation Taille finale du modèle
Modèles comparés	Cellpose Watershed Mask R-CNN YOLO 26m YOLO OBB 26m
Critère de sélection final	Meilleur compromis temps d'inférence / qualité de la segmentation/ taille du modèle

Le pipeline de cette étude s'est déroulé comme suit :

- Constitution et annotation d'un jeu de données dédié : 300 images de lots de cailloux ont été collectées et ensuite annotées via l'outil roboflow, celle-ci offrant à la fois un outil d'annotation simple d'utilisation et des fonctionnalités permettant d'accélérer le processus d'annotation.

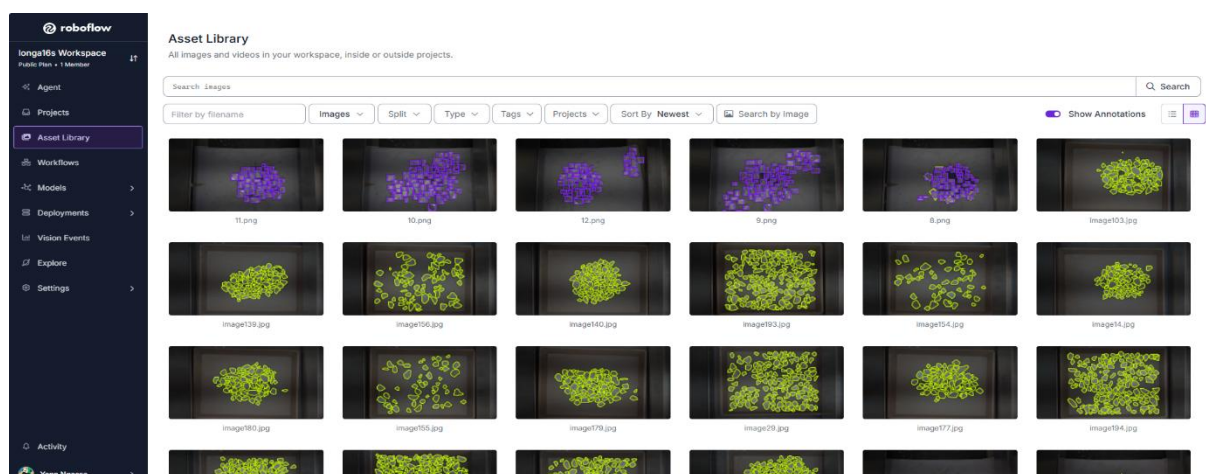


Figure 10: Dataset annoté avec l'outil Roboflow

Pour le cas du modèle YOLO OBB 26m nous avons procédé à une étape supplémentaire ne pouvant être entraînée sur des bounding boxes classiques mais sur des boîtes englobantes orientées car la sortie d'un détecteur d'objets orienté est un ensemble de boîtes englobantes

orientées qui entourent précisément les objets dans l'image, ainsi que des étiquettes de classe et des scores de confiance pour chaque boîte. Les boîtes englobantes orientées sont particulièrement utiles lorsque les objets apparaissent sous différents angles, comme dans l'imagerie aérienne, où les boîtes englobantes traditionnelles alignées sur les axes peuvent inclure des arrière-plans inutiles. Pour cela nous avons donc dû procéder comme suis :

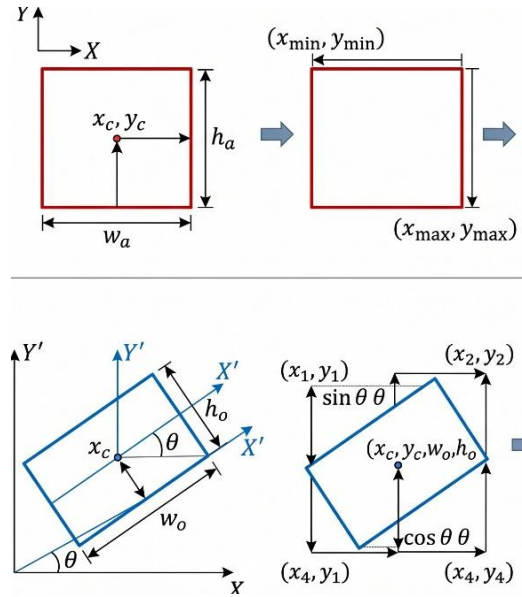


Figure 12: différence entre les types boîtes

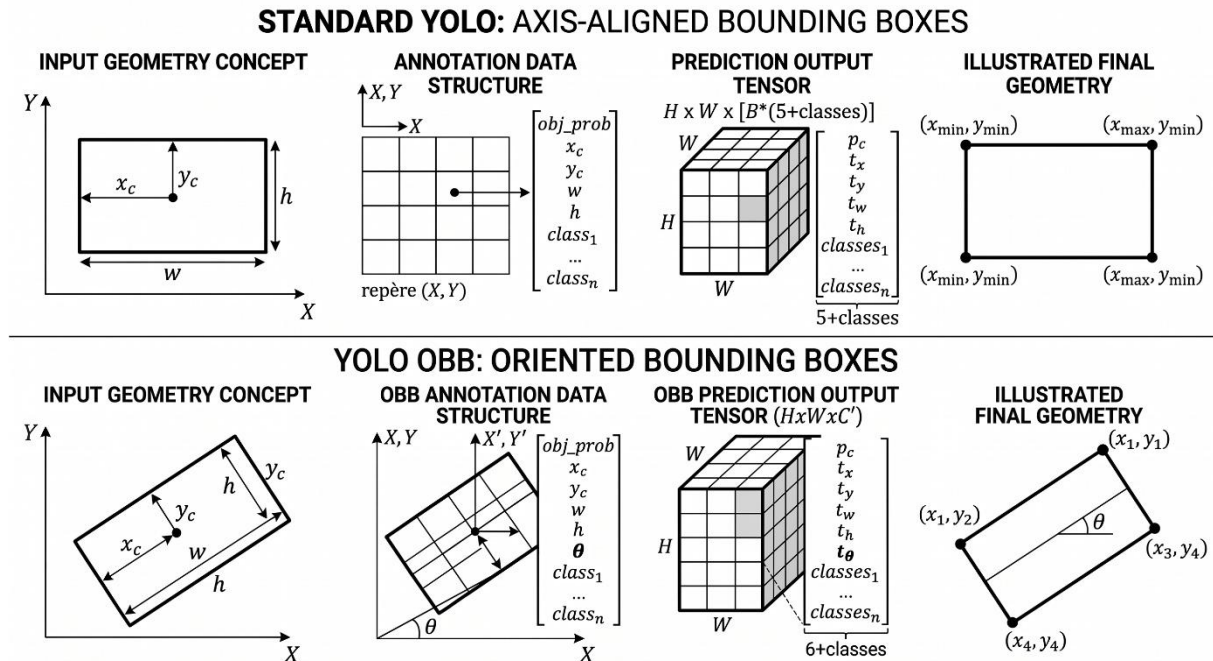


Figure 11: conversion des coordonnées des boîtes vers des boîtes orientées

La dénormalisation lorsqu'un dataset est annoté les annotations sont normalisées donc comprises entre 0 et 1 qui ressemble à ceci $[\text{class_id}, cx, cy, w, h]$ où cx et cy représentent les coordonnées du centre de la boîte, w et h sa largeur et sa hauteur. Ce format est pratique pour l'entraînement, mais le modèle SAM que l'on utilise ensuite ne comprend pas les coordonnées normalisées ; il attend des pixels absolus, c'est-à-dire des positions réelles dans l'image. Pour dénormaliser on calcule donc les 04 bords de la boîte, comme YOLO donne le centre et non les coins, il faut écarter les bords du centre d'une demi-largeur à gauche et à droite et d'une de mi-hauteur en haut et en bas

$$\begin{aligned}
x_{\min} &= \left(c_x - \frac{w}{2}\right) \times \text{largeur_image} \\
y_{\min} &= \left(c_y - \frac{h}{2}\right) \times \text{hauteur_image} \\
x_{\max} &= \left(c_x + \frac{w}{2}\right) \times \text{largeur_image} \\
y_{\max} &= \left(c_y + \frac{h}{2}\right) \times \text{hauteur_image}
\end{aligned}$$

On obtient donc une boîte exprimée en coordonnées réelles dans l'image dans le format $[x_{\min}, y_{\min}, x_{\max}, y_{\max}]$

Segmentation SAM Une fois que l'on dispose des coordonnées absolues de la boîte, on fait appel au modèle SAM de Meta AI. Le but ici est d'obtenir le masque des cailloux sur l'image à partir des coordonnées des différents bounding boxes, ce masque étant beaucoup plus riche qu'un bounding box précisément ce que l'on veut exploiter ici

Extraction du rectangle orienté depuis le masque On dispose maintenant d'un masque précis. L'objectif de cette dernière étape est d'en extraire un rectangle orienté. On commence par analyser le masque pour en extraire les contours. A partir de ces points de contour, on applique la fonction `cv2.minAreaRect` qui résout le problème de géométrie suivant : Parmi tous les rectangles possibles à n'importe quel angle qui peuvent contenir l'ensemble des points du contour, lequel a la surface la plus petite ? La réponse est calculée grâce à l'algorithme des calipers rotatifs, qui teste successivement un rectangle aligné sur chaque côté de l'enveloppe convexe de l'objet. La fonction retourne trois informations: le centre du rectangle, ses dimensions (largeur et hauteur), et son angle d'inclinaison.

Renormaliser les coordonnées La dernière opération consiste à renormaliser les coordonnées des 4 coins pour qu'elles soient à nouveau comprises entre 0 et 1, en les divisant par les dimensions de l'image. On obtient ainsi la ligne de label final au format YOLO OBB.

$[\text{Class_id}, x_1, y_1, x_2, y_2, x_3, y_3, x_4, y_4, w, h, \theta]$

- Répartition du jeu de données Train/Test split : Le jeu de données a été reparti en un sous-ensemble d'entraînement de 240 images et un sous-ensemble de validation de 60 images conformément à une répartition standard pour ce type de tâche, permettant d'évaluer la capacité de généralisation des différents modèles sur des images non vues pendant l'apprentissage.
- Entraînement des modèles : Chacun des modèles a été entraîné sur le sous-ensemble d'entraînement puis évalué sur le sous-ensemble de validation.

Tableau 4: Tableau comparatif de l'entraînement des différents modèles

Modèles	Temps d'entraînement total (min)	Consommation mémoire GPU (Go)	Batch	Nombre d'epoch	Learning rate
YOLO 26m	30	6.7	8	150	0.1
Mask R-CNN	45	7.1	8	150	0.1
YOLO OBB 26m	40	6.9	8	150	0.1

- Remplacement dans le pipeline applicatif de Cellpose par YOLO-OBb entraîné sur notre dataset, avec adaptation du code pour exploiter la sortie spécifique de YOLO-OBb (polygone à quatre sommet) en lieu et place des masques pixel par pixel produits par Cellpose.
- Validation finale sur données réelles et bascule complète de l'application principale sur le nouveau modèle.

Sur le plan de l'implémentation, le passage à YOLO-OBb a nécessité une adaptation du code de traitement d'image en aval de la détection, la nature de la sortie du modèle étant différente de celle de Cellpose. Là où Cellpose fournit directement un masque d'étiquetage pixel par pixel, YOLO-OBb fournit pour chaque objet détecté, un polygone orienté par ses 04 sommets. Le pipeline développé reconstruit donc un masque d'étiquetage équivalent à celui de Cellpose, par rastérisation de chaque polygone détecté, avant de réutiliser exactement la même chaîne d'analyse morphologique en aval, ce qui a permis de limiter la portée du changement au seul module de détection sans avoir à réécrire l'ensemble du pipeline de calcul granulométrique.

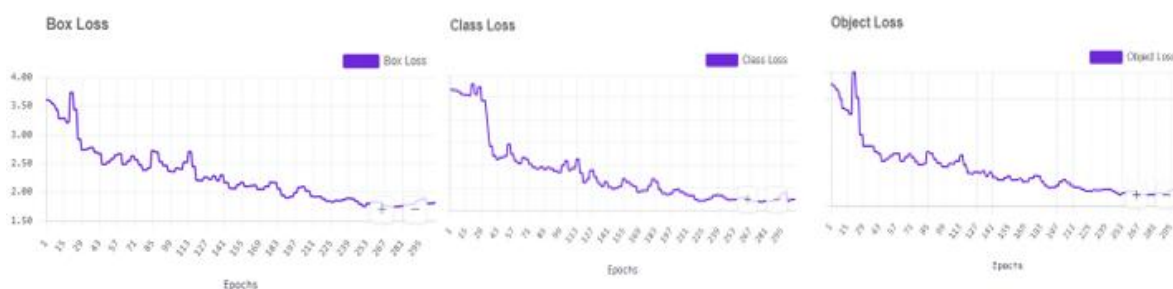


Figure 13: Courbe d'évolution de l'entraînement du modèle YOLO-OBb

Ce choix du modèle YOLO-OBb a été conforté par les résultats obtenus lors des entraînements menés sur le jeu de données annoté : la variante YOLO-OBb est beaucoup plus légère que le modèle Cellpose qui est une métrique importante pour notre logiciel étant donné qu'il embarquera le modèle retenu. YOLO-OBb a de meilleures performances de détection que les configurations à boîtes

alignées sur les métriques suivies (précision et rappel de détection, qualité d'ajustement des boîtes aux contours réels des particules) en plus de conserver l'avantage de rapidité d'inférence mis en évidence au tableau. C'est donc sur la base conjointe de ces résultats d'entraînement et des mesures de performance que la variante YOLO-OBb a été définitivement retenue comme modèle de segmentation pour l'application CIMES.

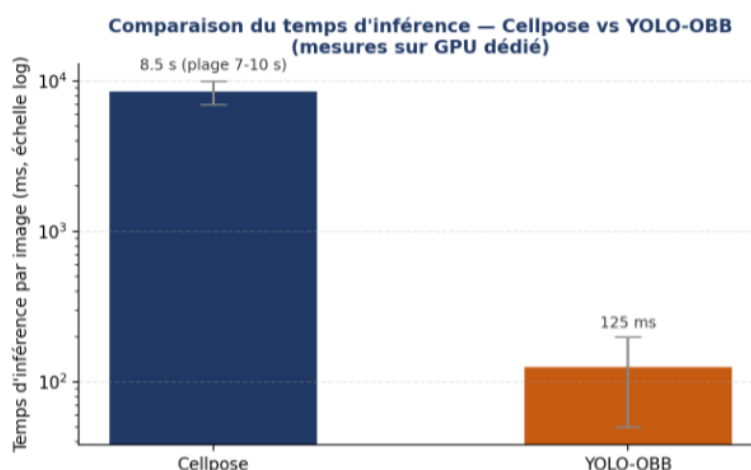


Figure 14: Comparaison du temps d'inférence de Cellpose et de YOLO-OBb

4.3 Implémentation du système de licence

Le logiciel étant destiné à être vendu et déployé sur des postes clients potentiellement dépourvus d'accès internet permanent, un système de protection par licence entièrement hors-ligne et basé sur la cryptographie asymétrique a été conçu et implémenté. Il répond aux exigences suivantes :

- Lier une licence à une ou plusieurs machines physiques spécifiques.
- Garantir l'intégrité de la licence.
- Contrôler le nombre de postes autorisés et la date d'expiration.
- Fonctionner sans connexion internet au démarrage de l'application.

Tableau 5: Exigences fonctionnelles et contraintes du système de licence

Exigence	Formulation
Hors-ligne	Le contrôle de licence doit s'exécuter sans aucune requête réseau vers un serveur distant, le poste client pouvant être physiquement isolé d'internet
Liaison	Une licence délivrée pour un client ne doit pas pouvoir être copiée et est destinée à un ou plusieurs postes spécifiques
Durée et volume	L'éditeur doit pouvoir limiter une licence dans le temps et le nombre de postes simultanément autorisés
Intégrité de la licence	Un utilisateur disposant d'un accès complet au binaire ne doit pas pouvoir fabriquer lui-même une licence valide.

4.3.1 Architecture du système

Le système repose sur 02 acteurs et un cycle de vie en 05 phases, schématisés ci-dessous. CIMES est le seul détenteur de la clé privée et seule habilité à émettre une licence, le poste client se contente de vérifier à chaque démarrage de l'application la validité d'une licence qui lui a été remise.

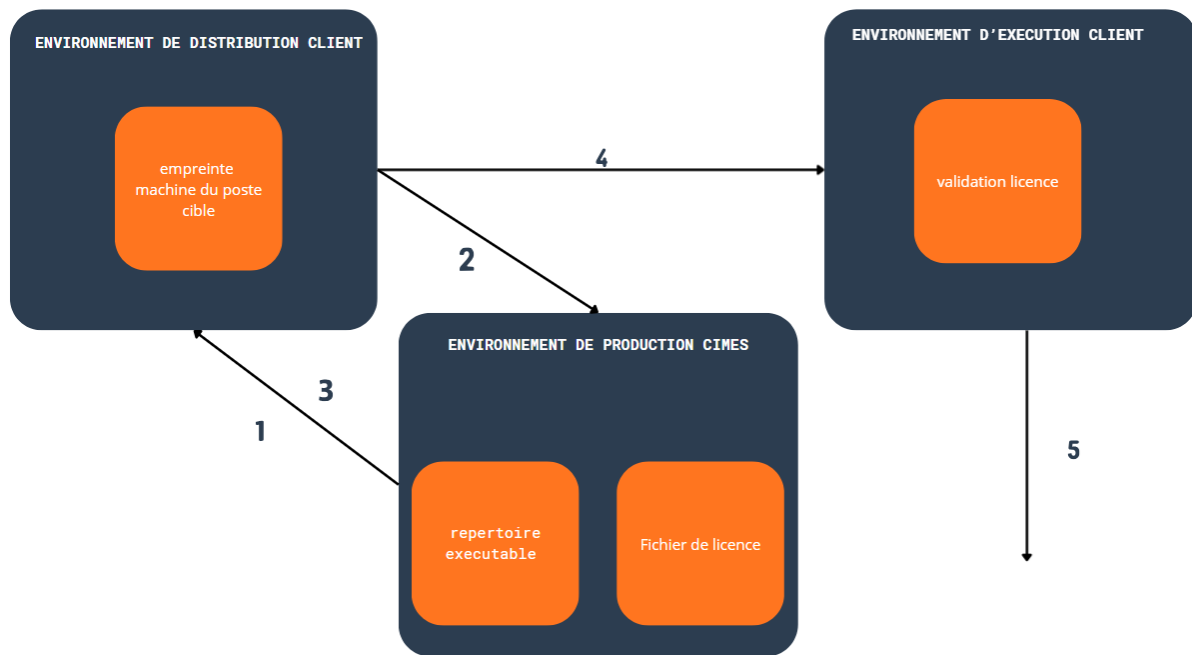


Figure 15 système de gestion des licences

1 : Le dossier zip contenant l'application principale, le guide d'installation et l'application de création de l'empreinte machine est envoyé au client.

2 : le client renvoie à l'équipe Cimes l'empreinte machine calculée sur la machine cible, poste sur lequel ils souhaitent déployer l'application.

3 : Les équipes de Cimes génèrent donc le fichier de licence associé à ce poste avec une date d'expiration pour cette dernière et envoie ce fichier au client.

4 : le client récupère le fichier de licence et le met dans le même répertoire que l'exécutable de l'application principale.

5 : l'application est lancée et le processus de validation est effectué en arrière et le client peut exploiter le logiciel.

La clé privée n'étant pas embarquée dans l'exécutable distribué, le binaire ne contient que la clé publique et la logique de vérification. Les 04 phases du cycle ici sont la génération de la paire de clé RSA, le calcul de l'empreinte matérielle du poste cible, l'émission de la licence et la vérification à chaque démarrage de l'application sur le poste du client.

4.3.2 Implémentation du système

- Génération de la paire de clé RSA : Une paire de clé de 2048 bits est générée coté éditeur. La clé privée conservée secrètement sert exclusivement à signer numériquement le contenu des fichiers de licence lors de leur émission, la clé publique quant à elle est embarquée en dur dans le code source de l'application cliente et ne permet que la vérification des signatures jamais leur génération. Cette asymétrie garantit qu'aucune licence valide ne peut être falsifiée même en disposant d'un accès complet au code source distribué. Le fichier de licence distribué est un document JSON contenant l'identité du client, l'empreinte matérielle du poste du client, la date d'expiration de la licence ainsi que la signature RSA calculée sur l'ensemble de ces données.

- Calcul de l’empreinte matérielle : Elle est obtenue en collectant tout d’abord 03 informations sur le poste (l’UUID du disque dur, le hostname et le modèle du processeur), ces 03 informations sont ensuite concaténées en une seule chaîne de texte, cette chaîne est ensuite passée dans une fonction de hachage SHA-256 celle-ci la transforme en une suite de 64 caractères hexadécimaux complètement différents de l’entrée. La dernière étape du processus consiste à prendre les 16 premiers caractères du hash et à les découper en 04 groupes de 04 séparés par des tirets on obtient ainsi l’empreinte finale au format lisible. Cette approche a pour avantage de produire toujours la même empreinte pour un poste de travail et 02 machines différentes ne produiront jamais la même empreinte.
- L’émission de la licence : Le fichier de licence distribué est un document JSON il est généré par CIMES et est signé avec la clé privée pour assurer son intégrité. Les clés de du dictionnaire JSON sont systématiquement triées par ordre alphabétique avant d’être signées ; puis à nouveau triées de la même manière lors de la vérification. Ceci garantit qu’une même licence produit toujours exactement la même chaîne de caractère à signer, indépendamment de l’ordre dans lequel les champs auraient pu être écrits manuellement.
- Vérification de la licence : Au lancement de l’application une validation en 05 étapes strictes est réalisée. Pour commencer la localisation du fichier .lic dans le répertoire de l’application s’il est présent on passe à la seconde étape qui est l’extraction de la signature et sérialisation canonique des données, afin de reconstituer exactement la chaîne signée par l’éditeur ensuite une vérification cryptographique de la signature RSA au moyen de la clé publique et la dernière étape consiste en une vérification que l’empreinte matérielle calculée localement figure bien dans le fichier de licence. Si l’une des étapes échoue la licence est rejetée dans son intégralité.

4.4 Intégration des sous-modules

Une fois validées indépendamment les applications « app-calibrage » et « Reglage-parametre » ont été rattachées au dépôt principal sous forme de sous-module Git, déclarés dans le fichier .gitmodules. Cette intégration a nécessité un travail d’adaptation qui implique : une harmonisation des chemins d’import relatifs entre le contexte d’exécution autonome de chaque module de calibration et l’application et exposition des deux modules comme option de lancement dédiées dans le point d’entrée principal.

Cette approche par sous-modules a permis de conserver, même après intégration la possibilité de lancer chaque outil indépendamment tout en garantissant que l’application principale embarque une version figée et testée de chaque module, point crucial pour la stabilité d’un produit industriel packagé en exécutable autonome.

4.5 Packaging de l’application et déploiement

La dernière étape de la chaîne de production logicielle consiste à transformer le code source en un exécutable autonome, installable et exploitable sur un poste de travail. Le cas échéant sans accès internet. Cette contrainte de déploiement s’applique au packaging.

4.5.1 Choix de l'outil de packaging

L'outil Pyinstaller a été retenu pour la génération de l'exécutable en cohérence avec l'écosystème Python déjà en place et la nécessité d'embarquer des dépendances volumineuses et hétérogènes. Contrairement à des alternatives comme Cx_Freeze ou Nuitka cet outil offre une prise en charge mature des hooks de collecte gratuite de dépendances et fonctionnalités indispensables ici compte tenu du grand nombre de fichier annexes que nécessitent Ultralytics et Pytorch pour fonctionner hors de leur environnement d'installation habituel.

4.5.2 Fichier de spécification

La configuration du build est centralisée dans un fichier de spécification, qui décrit explicitement les ressources à embarquer, les imports non détectés automatiquement par l'analyse statique de Pyinstaller et le point d'entrée de l'application.

4.5.3 Packaging de l'outil autonome d'identification machine

Un second exécutable est généré séparément par un script dédié qui invoque également Pyinstaller, mais sur le seul module de calcul d'empreinte matérielle, indépendamment du reste de l'application. Cet outil isolé répond à un besoin opérationnel précis côté client : Obtenir l'identifiant unique du poste de travail sur lequel sera installé le logiciel, à transmettre à l'équipe CIMES pour l'émission de la licence. Ce choix illustre un principe d'ingénierie logicielle classique qui est d'isoler dans un exécutable minimal une fonctionnalité utilisée dans un contexte distinct de celui de l'application principale.

4.5.4 Structure de livraison et procédure d'installation

Le résultat du build est un dossier autonome CIMES/ contenant l'exécutable, l'ensemble des dépendances et les ressources embarquées. Ce dossier constitue l'unité de livraison distribuée au client, accompagné d'un guide d'utilisation illustré détaillant la procédure d'installation, la structure de l'archive livrée et la procédure d'activation de la licence.

La procédure de déploiement chez un client type suit les étapes suivante : Envoie de répertoire zippé sur le poste cible, exécution de machine_id.exe pour obtenir l'empreinte matérielle du poste, transmission de cette empreinte à l'équipe CIMES en vue de l'émission de la licence, réception et dépôt du fichier .lic dans le répertoire de l'application par le client enfin le lancement de l'application principale déclenchant la vérification de licence puis en cas de succès l'ouverture de l'interface principale.

4.6 Résultats obtenus

Sur le plan fonctionnel, l'ensemble des briques décrites ci-dessus a été développé, testé et intégré avec succès dans l'application CIMES.

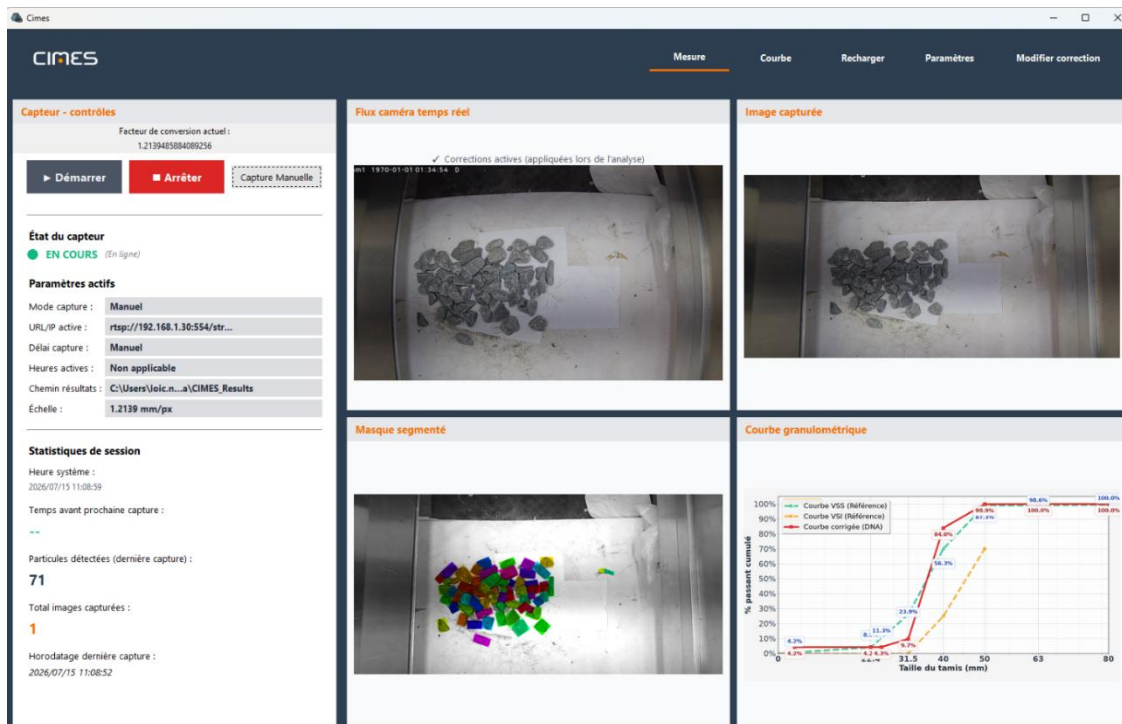


Figure 16 rendu final de l'interface principale

Les principales fonctionnalités tel que la segmentation par YOLO, le pipeline complet d'analyse des images et du tracé des courbes granulométriques, génération des rapports PDF, transmission automatique par courriel électronique et le système de licence RSA fonctionnel de bout en bout.

4.6.1 Ergonomie et expérience utilisateur

Au-delà des aspects purement techniques, une attention particulière a été portée à l'ergonomie des interfaces développées. Celles-ci étant destinées à être utilisées par des opérateurs de terrain. Un système de thèmes centralisé définit une charte graphique cohérente, des polices et des constantes d'espacement partagées assurant une identité visuelle homogène entre les différents outils du produit malgré leur développement initialement indépendant.

Un soin particulier a également été apporté à la gestion des erreurs et aux messages utilisateur en effet la quasi-totalité des fonctions critiques du cœur métier intercepte explicitement les exceptions susceptibles de survenir et les traduit en messages explicites affichés à l'utilisateur plutôt que de laisser l'application se terminer brutalement.

4.6.2 Stratégie de tests

La stratégie de test retenue a combiné plusieurs niveaux de vérification, au niveau unitaire, certains modules de calcul ne dépendant d'aucune ressource externe ont pu être testés de manière isolée à l'aide de jeux de données synthétiques et de notebook jupyter dédiés, permettant de vérifier la cohérence des sorties pour des cas limités.

Au niveau fonctionnel, chaque application autonome a été testée manuellement sur des jeux d'images et de mesures réels fournis par l'entreprise, avant intégration. Dans le système complet des campagnes de test end-to-end ont été menées sur le flux de vidéo, courant le cycle complet des fonctionnalités contenu dans l'application ainsi que le cycle complet du système de licence.

Chapitre 5: Analyse critique et discussion

Ce chapitre présente une auto-critique des résultats obtenus, des limites du livrable sur les plans méthodologique, technique et les difficultés rencontrées durant la phase de développement.

5.1 Bilan des forces et des choix validés

Les développements réalisés répondent globalement aux objectifs fixés en début de stage. L'architecture modulaire retenue a démontré sa pertinence et la décomposition du projet en sous-modules autonomes a permis de développer, déboguer et valider chaque brique en parallèle, ceci réduisant drastiquement le risque de régression lors des intégrations. Cette structuration facilitera également la maintenance ultérieure et l'embarquement de nouvelles fonctionnalités par les équipes de CIMES.

Le choix du modèle YOLO-OBBS s'est révélé particulièrement adapté à la géométrie irrégulière des granulats. Comparativement à Cellpose, la réduction du temps d'inférence a rendu le système compatible avec une supervision continue en production. La conservation de la chaîne d'analyse en aval a permis une transition technique fluide sans remise en cause des algorithmes de calcul granulométrique préexistants.

Le système de licence bien que technique, a pu être conçu et validé de bout en bout dans le temps imparti. Le recours au standard RSA-2048/PSS/SHA-256, éprouvé et documenté offre un niveau de sécurité jugé satisfaisant pour un déploiement ciblé de logiciel métier répondant à la contrainte impérative de fonctionnement hors-ligne.

5.2 Limites et difficultés rencontrées

Plusieurs limites méthodologiques et techniques peuvent être soulignées. Elles peuvent être propres aux choix de conception et d'hypothèses de travail retenus, aux caractéristiques intrinsèques des technologies mobilisées et les difficultés opérationnelles rencontrées au cours du développement qui n'ont pas nécessairement donné lieu à des limites persistantes mais ont représentés des défis significatifs à surmonter dans le temps imparti du stage.

5.2.1 Limites méthodologiques

La première limite méthodologique porte sur le protocole de comparaison des modèles de segmentation. Si ce protocole a permis d'objectiver la décision de changement de Cellpose vers YOLO-OBBS sur le critère du temps d'inférence, la métrique de qualité de détection retenue est restée en grande partie qualitative soit une évaluation visuelle du chevauchement des masques avec les particules réelle, faute de temps pour mettre en place des métriques quantitatives standardisées de la littérature sur un jeu de test annoté dédié à cette évaluation. Le choix final reste donc solidement étayé sur le plan de la performance temporelle, mais plus approximatif sur le plan strictement métrologique de la qualité de détection. La seconde limite concerne le modèle de correction. Celui-ci repose sur deux paramètres globaux appliqués de manière uniforme sur l'ensemble de la plage de tailles de particules, sous l'hypothèse qu'une transformation affine par morceaux suffit à recaler la courbe optique sur la courbe de tamisage réel. Cette hypothèse, vérifiée dans la plage de tamis expérimentaux couverte par les données de calibration disponibles, ne capture pas nécessairement des écarts non linéaires plus complexes qui pourraient apparaître pour des granulométries extrêmes ou pour des matériaux

de nature différente de ceux utilisés lors du réglage initial. Une correction par segments plus fins, voire un modèle de correction appris statistiquement sur un plus grand nombre de campagnes de mesure, pourrait affiner ce recalage, mais n'a pas pu être exploré dans le temps imparti.

Enfin, la démarche de validation retenue pour l'ensemble du projet a privilégié des tests fonctionnels et des comparaisons empiriques sur des données réelles, au détriment d'une couverture de tests automatisés exhaustive comme des tests unitaires systématiques sur l'ensemble des modules et des tests d'intégration continue couvrant l'intégralité du pipeline. Ce choix, cohérent avec les contraintes de temps d'un stage et avec la nature exploratoire du projet, constitue néanmoins une limite du point de vue de la maintenabilité à long terme du logiciel, qui devra être comblée après la fin du stage pour sécuriser les évolutions futures du produit.

5.2.2 Limites techniques

Sur le plan technique, la robustesse du modèle YOLO-OBB dépend directement de la représentativité du jeu d'images d'entraînement. Des conditions d'éclairage inhabituelles, un objectif de caméra encrassé ou poussiéreux, ou des types de matériaux granulaires non représentés dans ce jeu de données, pourraient dégrader la qualité de détection en production sur un nouveau site client, sans qu'il soit possible de le garantir formellement en l'absence d'une campagne de validation plus large couvrant une diversité de conditions industrielles.

Le mécanisme d'empreinte matérielle bien que fonctionnellement efficace, présente une fragilité opérationnelle qui est le remplacement d'un composant matériel identifié ce qui modifie l'empreinte et invalide la licence, imposant une intervention coté éditeur pour émission. De plus l'absence de mécanisme de révocation en ligne, conséquence directe de la contrainte hors-ligne empêche l'invalidation à distance d'une licence compromise ou volée. Enfin la clé publique embarquée dans le binaire, bien qu'insuffisante pour forger une signature, pourrait théoriquement faire l'objet d'une ingénierie inverse visant à patcher la vérification dans le code compilé.

Enfin, l'architecture logicielle retenue pour le packaging génère un exécutable dont le poids final est significativement alourdi par l'inclusion complète de Pytorch et d'Ultralytics, ce qui allonge le temps de premier démarrage de l'application et la taille de l'archive à distribuer aux clients, contrainte technique inhérente à l'usage d'un Framework d'apprentissage profond généraliste plutôt que d'un moteur d'inférence allégé, piste qui n'a pas pu être explorée durant le stage faute de temps.

5.2.3 Difficultés rencontrées

Au-delà des limites persistantes ci-dessus, plusieurs difficultés concrètes ont dû être surmontées en cours de projet. La plus significative a concerné le packaging de l'exécutable final avec PyInstaller. En effet l'inclusion correcte des bibliothèques dynamiques de Pytorch nécessaires à l'inférence YOLO s'est heurtée à des échecs de chargement de DLL une fois l'application figée en exécutable, un problème dont le diagnostic a nécessité plusieurs itérations de compilation et de test avant d'aboutir à la solution retenue.

L'intégration des deux sous-modules Git au sein de l'application principale a également constitué une difficulté non anticipée initialement car ces deux outils ayant été développés et testés de manière autonome, avec leurs propres conventions d'import et leur propre structure de projet, leur rattachement en tant que sous-modules a nécessité un travail de réconciliation des

chemins d'import relatifs et de la configuration partagée, afin d'éviter toute divergence entre le comportement d'un module en exécution autonome et son comportement une fois intégré.

La constitution du jeu de données d'entraînement YOLO-OBB s'est également révélée plus chronophage qu'anticipé, l'annotation manuelle de boîtes orientées nécessitant de positionner quatre sommets par particule plutôt qu'un simple rectangle aligné, ceci demandant davantage de rigueur et de temps par image que l'annotation en boîtes classiques, sans outil d'assistance automatique à l'annotation disponible en début de projet.

Enfin, le réglage des paramètres de correction a nécessité plusieurs itérations empiriques avant la mise en place de la fonctionnalité d'optimisation, les premiers essais de réglage manuel, réalisés par tâtonnement à partir de l'interface graphique de Réglage-parametre, ont mis en évidence la nécessité de disposer d'une métrique d'erreur quantitative (pour objectiver la qualité du recalage), plutôt que de se fier à une simple appréciation visuelle de la superposition des courbes.

Ces difficultés, une fois résolues, ont chacune donné lieu à une solution documentée dans le code via une gestion défensive des erreurs, des messages explicites, des mécanismes automatisés remplaçant les réglages manuels et contribuant à la robustesse et à la maintenabilité finales du logiciel livré.

5.3 Discussion sur la chaîne de mesure

La traçabilité métrologique constitue le nerf de la guerre dans tout système de mesure industrielle. Dans le cas présent, la chaîne de mesure s'appuie sur une succession de transformations. Chaque maillon introduit une incertitude qui se propage et se combine. Le tamisage mécanique, bien que lui aussi soumis à des variabilités (homogénéité de l'échantillon, usure des mailles, technique de vibration), bénéficie d'une traçabilité normative établie et d'une reconnaissance réglementaire. La vision par ordinateur, en revanche, constitue une méthode alternative qui doit démontrer sa corrélation à la référence. Les écarts observés entre la courbe optique brute et la courbe de tamisage de l'ordre de 10 à 25 % selon les classes granulométriques justifient pleinement l'existence du module de correction. Cependant, cette correction, calibrée sur un nombre limité de lots, reste un modèle de régression sans fondement mécanique explicite. Une voie d'amélioration consisterait à modéliser la relation projection-tamisage par une approche stéréométrique ou tomographique, capturant la troisième dimension des particules. En l'absence de telle infrastructure, le système actuel constitue un compromis pragmatique entre précision, coût et débit de traitement, pleinement justifié par le contexte industriel mais devant faire l'objet d'une vigilance métrologique continue.

Chapitre 6: Perspectives et bilan de l'expérience

Dans ce chapitre il est présenté les perspectives d'évolution du projet sur les plans technique et fonctionnelles.

6.1 Perspectives techniques et fonctionnelles

Le système développé ouvre plusieurs pistes d'évolution prometteuses, tant sur le plan technique que fonctionnel.

La principale limite métrologique identifiée réside dans la nature projective de la mesure 2D, l'intégration d'un système stéréoscopique avec 02 caméras permettrait de reconstruire la hauteur des particules et d'estimer leur volume améliorant considérablement la corrélation avec le tamisage mécanique qui mesure le plus petit côté de passage.



Figure 17: modèle d'image de lot de cailloux

Le modèle actuel est figé après son entraînement initial. Une infrastructure de réentraînement continu, alimentée par les images capturées en production et annotées semi-automatiquement permettra d'adapter le détecteur aux variations saisonnières de l'éclairement ou aux changements de fournisseurs de ballast.

Le système de licence purement hors-ligne pourrait évoluer vers un modèle hybride constitué d'une vérification locale obligatoire pour garantir le fonctionnement sans connexion complétée par une synchronisation périodique optionnelle avec un serveur de licences, permettant la révocation à distance, le suivi d'usage et le déploiement de mises à jour cryptées.

6.2 Bilan personnel

Ce stage de fin d'études au sein de CIMES m'a permis de confronter les connaissances théoriques acquises durant ma formation à la réalité d'un projet industriel complet, de la conception à la livraison. Sur le plan technique, j'ai consolidé mes compétences en vision par ordinateur tels que la segmentation d'instances, la calibration caméra et la métrologie optique,

en apprentissage profond (entraînement et déploiement de modèles YOLO), en cryptographie appliquée (implémentation RSA-PSS) et en ingénierie logicielle les pratiques de clean architecture, packaging et CI/CD. La refactorisation d'une base de code monolithique en architecture modulaire m'a sensibilisé aux enjeux de maintenabilité et de collaboration en équipe. Sur le plan méthodologique, la conduite d'un projet en cycles itératifs, avec des points de synchronisation réguliers et une validation continue par des tests fonctionnels sur données réelles, m'a appris à concilier rigueur scientifique et contraintes de délai. La rédaction de documentation technique et de guides utilisateur m'a également fait prendre conscience de l'importance de la transmission des savoirs dans un contexte industriel. Enfin, sur le plan humain, l'intégration dans une équipe pluridisciplinaire au sein du Technopole Transalley m'a permis de développer une capacité d'écoute des besoins métiers et d'adaptation aux contraintes opérationnelles des clients industriels. En conclusion, le système d'analyse granulométrique par vision artificielle développé durant ce stage démontre la faisabilité d'une métrologie optique corrélée aux standards normatifs de tamisage, tout en répondant aux impératifs de déploiement hors ligne et de protection logicielle propres à l'industrie ferroviaire. Les perspectives d'évolution, notamment en stéréovision et en modélisation physique de la correction, ouvrent la voie à une amélioration continue de la précision et de la robustesse du système. Ce projet constitue une contribution concrète aux activités de recherche et développement de CIMES, et une expérience formatrice pour ma future carrière.

Conclusion

Ce projet répondait à un besoin industriel précis qui est celui de développement d'un outil professionnel d'analyse granulométrique par vision artificielle destiné au contrôle qualité industriel du ballast ferroviaire. Il m'a permis de consolider les compétences en vision par ordinateur, apprentissage profond, traitement d'image et en analyses de données scientifiques acquises durant mon parcours académique. J'ai également mis en œuvre pour la première fois de manière opérationnelle un système de sécurisation logicielle par cryptographie asymétrique RSA, sujet qui était encore un peu flou pour moi mais essentiel dans un contexte de distribution commerciale de logiciel et de sécurisation de systèmes sensibles.

Sur le plan transversal, ce stage a été l'occasion de développer ma capacité à travailler en autonomie sur des développements longs et multi-modules, à documenter techniquement mon travail à destination d'autres développeurs ou de futurs utilisateurs, à dialoguer avec des tuteurs pour arbitrer entre exigences techniques et contraintes métier et à organiser mon travail selon une démarche itérative dans un contexte où les spécifications évoluaient au fil de l'avancement du projet.

Cette expérience s'inscrit en pleine cohérence avec mon parcours académique et avec mon projet professionnel, orienté vers l'intelligence artificielle et le machine Learning. Il m'a permis de confronter les connaissances théoriques acquises en formation à la réalité opérationnelle d'un développement produit destiné à être déployé et vendu à des clients, avec les contraintes de robustesse, de documentation et de protection commerciale que cela implique. Ce stage conforte mon souhait de poursuivre une carrière professionnelle dans le domaine de l'intelligence artificielle.

La diversité des tâches confiées, allant du développement d'interfaces graphiques métier jusqu'à la cryptographie appliquée en passant par l'entraînement et l'intégration d'un modèle d'apprentissage profond a également constitué une richesse pédagogique notable rarement réunie dans un seul et même projet académique et représentative de la polyvalence attendue d'un ingénieur informaticien évoluant au sein d'une entreprise où chaque collaborateur est amené à intervenir sur l'ensemble de la chaîne de valeur logicielle, de conception à la mise en production.

Au-delà du périmètre strict de ce stage, les problématiques traitées comme la segmentation d'objet trouvent des échos dans de nombreux autres secteurs au-delà de la granulométrie. Ceci montre que les compétences développées au cours de ce stage tant sur le plan de la vision par ordinateur que celui du génie logiciel industriel constituent ainsi un socle transférable à un large spectre d'applications de contrôle et de supervision par intelligence artificielle.

Bibliographie

- [1] Edmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. *CVPR*.
- [2] Bradski, G., & Kaehler, A. (2008). *Learning OpenCV: Computer Vision with the OpenCV Library*. O'Reilly Media.
- [3] Szeliski, R. (2022). *Computer Vision: Algorithms and Applications* (2e éd.). Springer.
- [4] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444.
- [5] Zhou, X., et al. (2019). IoU Loss for 2D/3D Object Detection. *International Conference on 3D Vision (3DV)*.
- [6] Zhang, Z. (2000). A flexible new technique for camera calibration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(11), 1330–1334.
- [7] Ultralytics Team. (2024). *Ultralytics Documentation — OBB (Oriented Bounding Boxes)*.
- [8] Virtanen, P., Gommers, R., Oliphant, T. E. et al. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272.
- [9] van der Walt, S., Schönberger, J. L., Nunez-Iglesias, J. et al. (2014). scikit-image: image processing in Python. *PeerJ*, 2:e453
- [10] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- [11] Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.
- [12] Rivest, R. L., Shamir, A., & Adleman, L. (1978). A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2), 120–126.
- [13] National Institute of Standards and Technology (NIST). (2015). *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions* (FIPS PUB 202).
- [14] Alan D. Moore Python GUI Programming with Tkinter, Design and build functional and user-friendly GUI applications, 2nd Edition
- [15] Python Cryptographic Authority (PyCA). (2024). *cryptography — Cryptographic Recipes and Primitives* ($\geq 49.0.0$).
- [16] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95
- [17] McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference (SciPy 2010)*, 51–56.
- [18] Hartmans, S., & Pietrek, M. (2023). *PyInstaller Manual*
- [19] Astral Software. (2024). *uv — An extremely fast Python package and project manager*.

Annexes

Annexes A : Code source partiel du projet

```
"""
Main entry point for the Cimes application
"""

import sys
import os
import tkinter as tk
import warnings
from tkinter import messagebox
from src.ui.main_app import CimesApp
from src.license import check_license, register_license, LicenseStatus
from src.license.ui import LicenseActivationDialog

# Redirection des logs vers un fichier uniquement si CIMES_DEBUG=1 est défini
# (jamais actif en production – évite de logger les données sensibles en clair)
if getattr(sys, "frozen", False) and os.environ.get("CIMES_DEBUG") == "1":
    log_dir = os.path.join(os.path.expanduser("~"), "AppData", "Local", "Cimes")
    os.makedirs(log_dir, exist_ok=True)
    log_path = os.path.join(log_dir, "debug.log")
    try:
        _log_file = open(log_path, "w", encoding="utf-8") # pylint: disable=consider-using-with
        sys.stdout = _log_file
        sys.stderr = _log_file
    except Exception: # pylint: disable=broad-except
        pass
    print("=== Démarrage de l'application ===")
    print(f"Executable: {sys.executable}")
    print(f"MEIPASS: {getattr(sys, '_MEIPASS', 'N/A')}")

# Gestion du dossier des DLL de torch dans l'exécutable
if getattr(sys, "frozen", False) and hasattr(sys, "_MEIPASS"):
    _MEIPASS = getattr(sys, "_MEIPASS", "")
    # Chemins possibles pour les DLLs de torch, dépend de onedir vs onefile
    torch_dll_paths = [
        os.path.join(_MEIPASS, "_internal", "torch", "lib"),
        os.path.join(_MEIPASS, "torch", "lib"),
    ]
    for torch_dll_path in torch_dll_paths:
        if os.path.exists(torch_dll_path):
            print(f"Ajout du dossier DLL torch: {torch_dll_path}")
            if hasattr(os, "add_dll_directory"):
                os.add_dll_directory(torch_dll_path)
            else:
                os.environ["PATH"] = torch_dll_path + os.pathsep + os.environ["PATH"]

# Import anticipé de torch pour stabiliser le chargement des DLLs
try:
    import torch

    print(f"Torch chargé avec succès (version {torch.__version__})")
```

1-Fichier main.py point d'entrée du projet

```

print(f"CUDA disponible: {torch.cuda.is_available()}")
except Exception as e: # pylint: disable=broad-exception
    print(f"Erreur lors du pré-chargement de torch: {e}")

warnings.filterwarnings("ignore")

if getattr(sys, "frozen", False) and hasattr(sys, "_MEIPASS"):
    # pylint: disable=protected-access
    sys.path.insert(0, sys._MEIPASS)
else:
    sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

if __name__ == "__main__":
    # — Gestion des modules secondaires
    if "--module-calibration" in sys.argv:
        from modules.app_calibrage_cam.ui.main_window import ApplicationCalibrage
        _mod_app = ApplicationCalibrage()
        _mod_app.mainloop()
        sys.exit(0)

    if "--module-correction" in sys.argv:
        from modules.app_change_corr_params.src.ui.main_window import CIMESApp
        _mod_app = CIMESApp()
        _mod_app.mainloop()
        sys.exit(0)

    # — Application principale

    # Vérification de la licence avant le lancement de l'application
    print("Vérification de la licence...")
    license_status: LicenseStatus = check_license()
    print(f"Statut licence : {license_status.message}")

    if not license_status.valid:
        def handle_activation(lic_filepath):
            """
            Installe et valide un fichier de licence (.lic) sélectionné par l'utilisateur.

            :param lic_filepath: Chemin absolu vers le fichier .lic sélectionné.
            :return: Tuple (valid: bool, message: str)
            """
            reg_status = register_license(lic_filepath)
            return reg_status.valid, reg_status.message

        dialog = LicenseActivationDialog(on_activate_callback=handle_activation)
        dialog.mainloop()

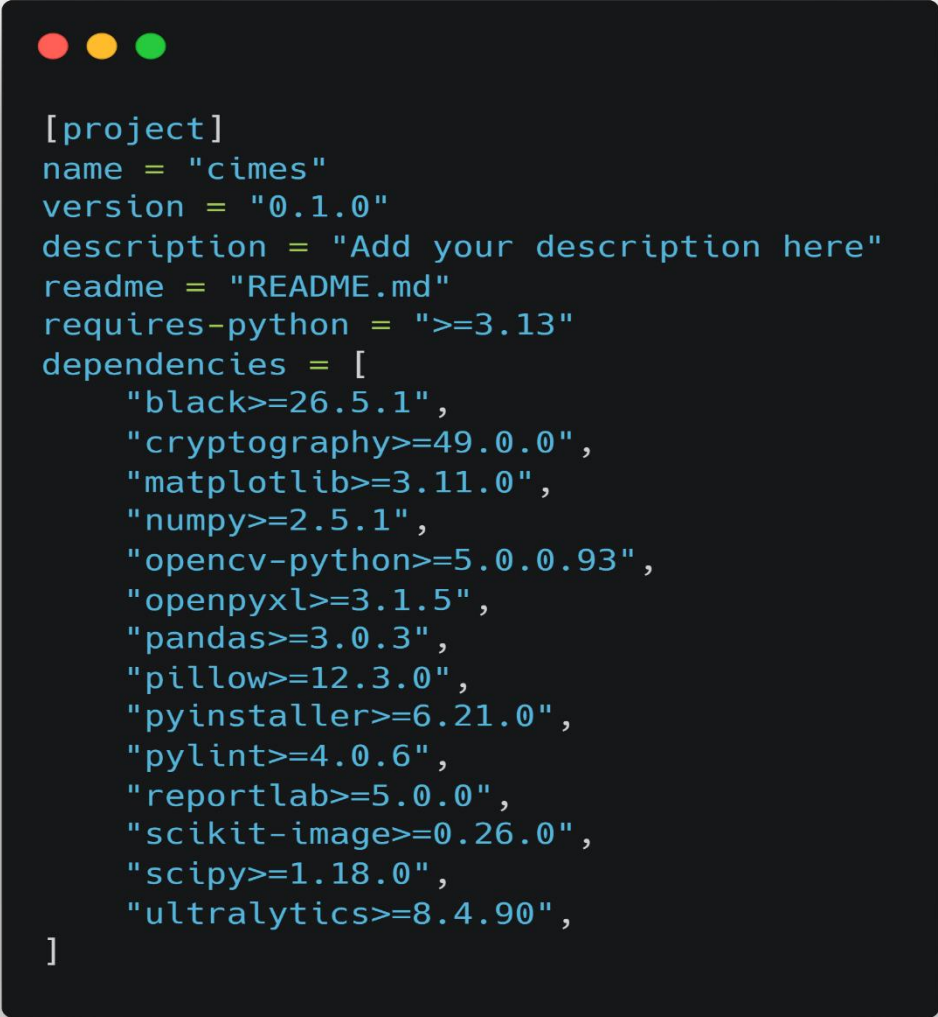
        if dialog.result:
            print("Licence activée avec succès.")
            license_status = check_license()
        else:
            print("Activation annulée ou échouée.")
            sys.exit(1)

    # Si la licence n'est toujours pas valide après tentative d'enregistrement
    if not license_status.valid:
        _root_tk = tk.Tk()
        _root_tk.withdraw()
        messagebox.showerror(
            title="Licence invalide - CIMES",
            message=(
                f"\u26a0\ufe0f Accès refusé\n\n"
                f"{license_status.message}\n\n"
                "Contactez votre administrateur pour renouveler votre licence."
            ),
        )
        _root_tk.destroy()
        sys.exit(1)

    try:
        app = CimesApp()
        app.mainloop()
    except Exception as e: # pylint: disable=broad-exception-caught
        print(f"Erreur : {e}")
        import traceback

        traceback.print_exc()
        input("Appuyez sur Entrée pour quitter...")

```



```
[project]
name = "cimes"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.13"
dependencies = [
    "black>=26.5.1",
    "cryptography>=49.0.0",
    "matplotlib>=3.11.0",
    "numpy>=2.5.1",
    "opencv-python>=5.0.0.93",
    "openpyxl>=3.1.5",
    "pandas>=3.0.3",
    "pillow>=12.3.0",
    "pyinstaller>=6.21.0",
    "pylint>=4.0.6",
    "reportlab>=5.0.0",
    "scikit-image>=0.26.0",
    "scipy>=1.18.0",
    "ultralytics>=8.4.90",
]
```

2- fichier pyproject.toml de configuration des dépendances du projet via uv

```
# -*- mode: python ; coding: utf-8 -*-
from PyInstaller.utils.hooks import collect_all

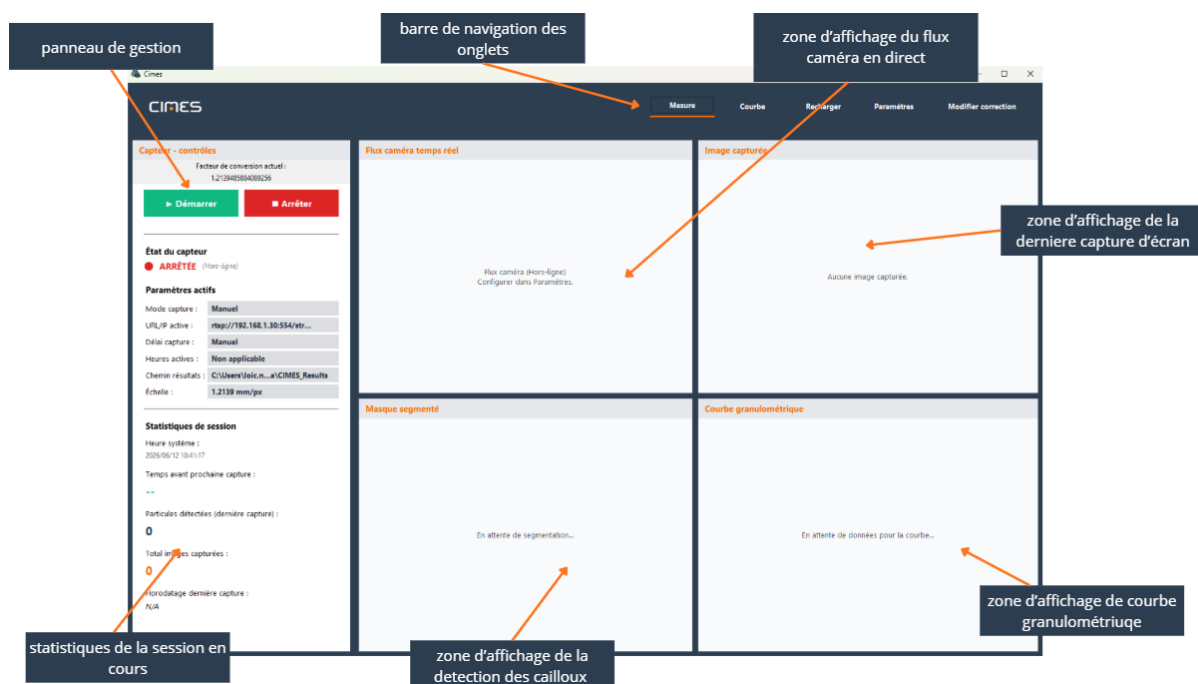
datas = [('assets', 'assets'), ('best.pt', '.'), ('mesure_config.json', '.'), ('modules', 'modules')]
binaries = []
hiddenimports = ['sklearn.utils._cython_blas', 'sklearn.neighbors._typedefs', 'sklearn.utils._heap',
'sklearn.utils._sorting', 'sklearn.utils._vector_sentinel', 'cryptography', 'cv2']
tmp_ret = collect_all('ultralitics')
datas += tmp_ret[0]; binaries += tmp_ret[1]; hiddenimports += tmp_ret[2]
tmp_ret = collect_all('timm')
datas += tmp_ret[0]; binaries += tmp_ret[1]; hiddenimports += tmp_ret[2]

a = Analysis(
    ['main.py'],
    pathex=[],
    binaries=binaries,
    datas=datas,
    hiddenimports=hiddenimports,
    hookspath=[],
    hooksconfig={},
    runtime_hooks=['runtime_hook_torch.py'],
    excludes=[],
    noarchive=False,
    optimize=0,
)
pyz = PYZ(a.pure)

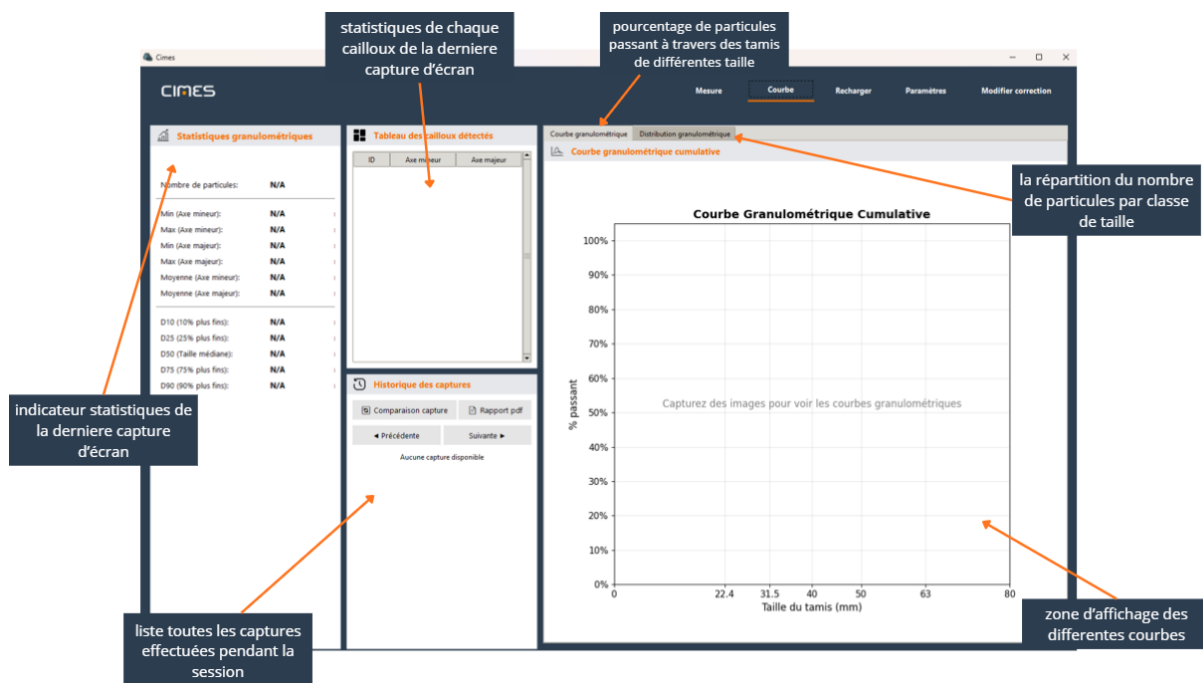
exe = EXE(
    pyz,
    a.scripts,
    [],
    exclude_binaries=True,
    name='App',
    debug=False,
    bootloader_ignore_signals=False,
    strip=False,
    upx=True,
    console=False,
    disable_windowed_traceback=False,
    argv_emulation=False,
    target_arch=None,
    codesign_identity=None,
    entitlements_file=None,
)
coll = COLLECT(
    exe,
    a.binaries,
    a.datas,
    strip=False,
    upx=True,
    upx_exclude=[],
    name='CIMES',
)
```

3- Fichier Cimes.spec de configuration pour la génération des différents executables.

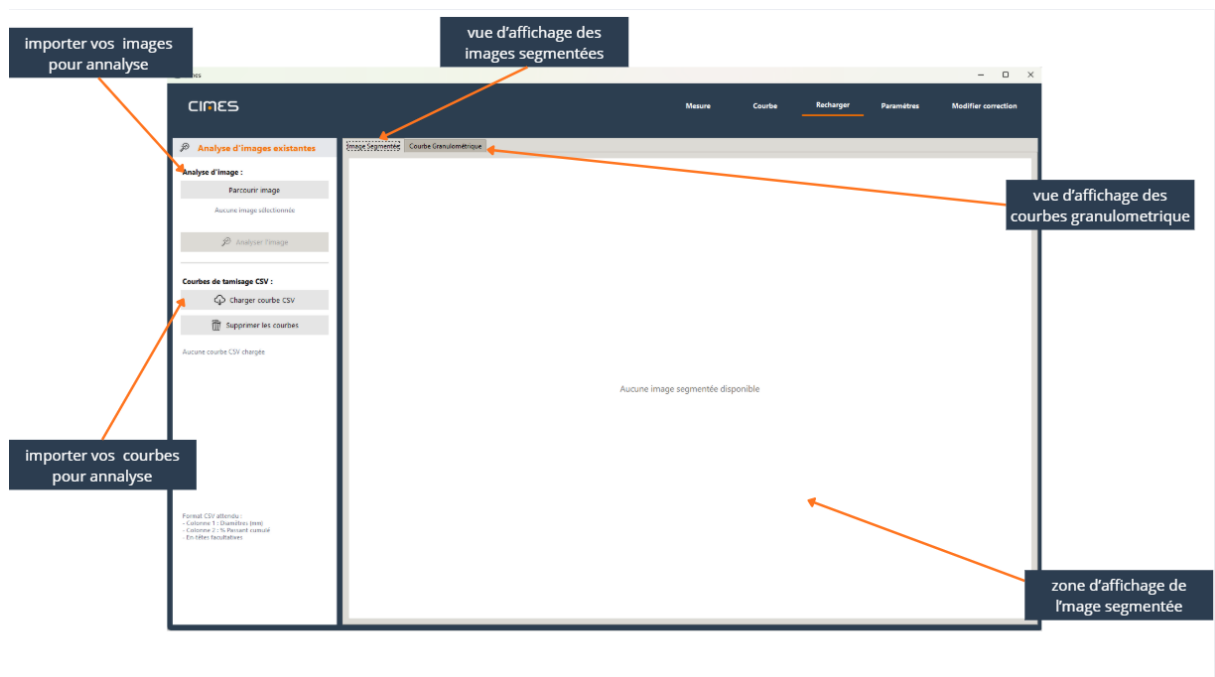
Annexes B : Images des différentes interfaces



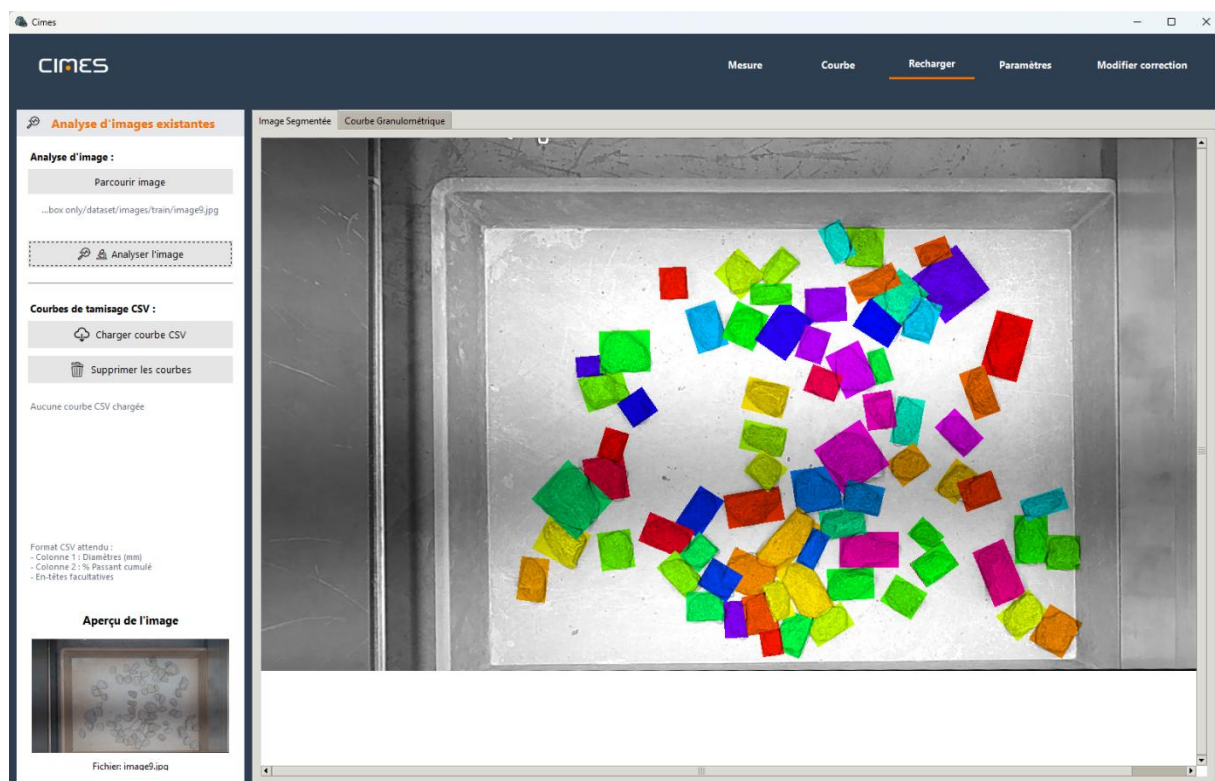
1- Interface principale du logiciel



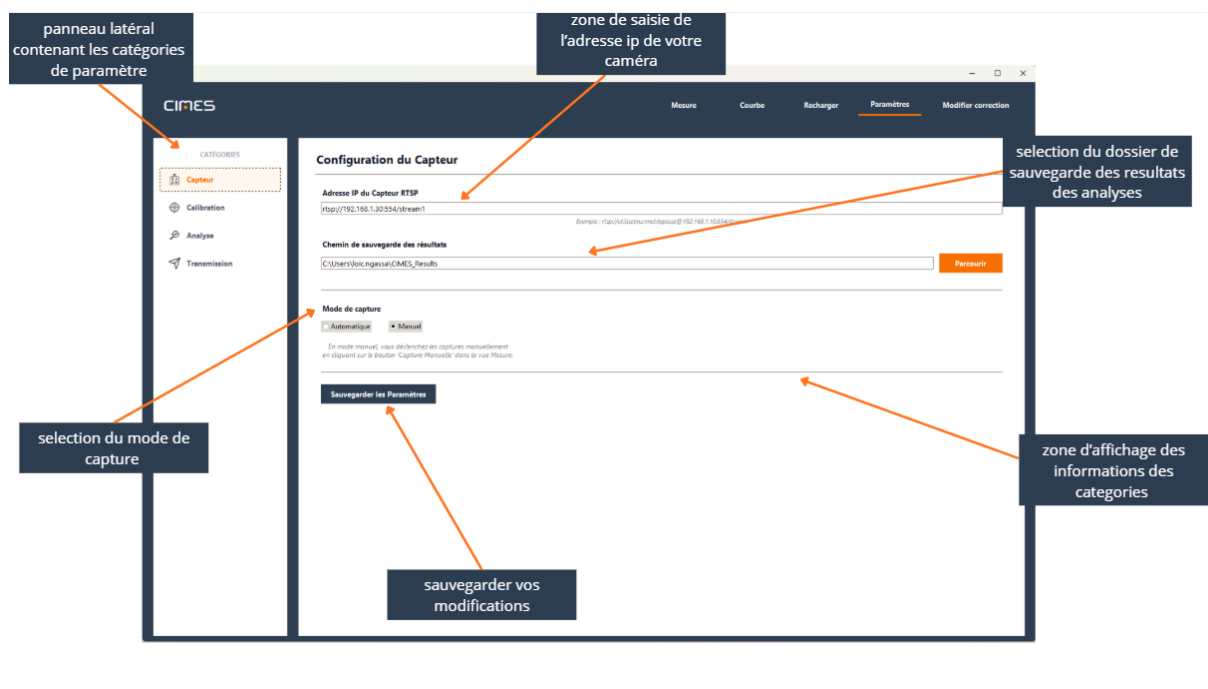
2- Interface « courbe » du logiciel



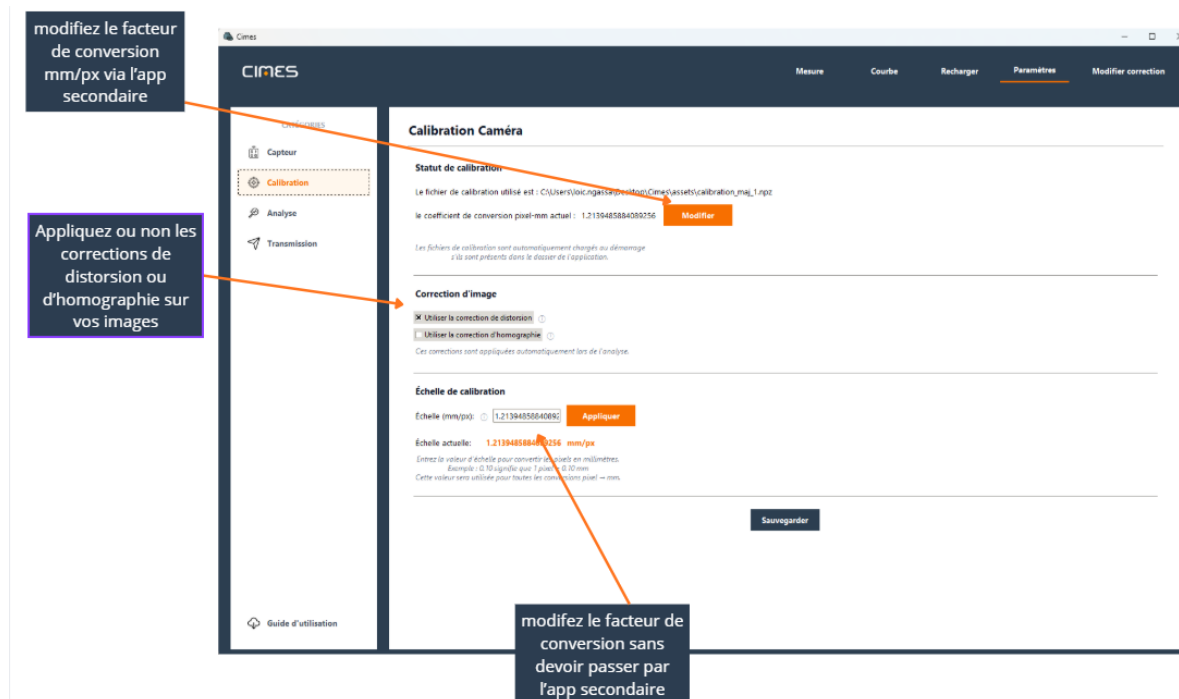
3- Interface « Recharge » du logiciel



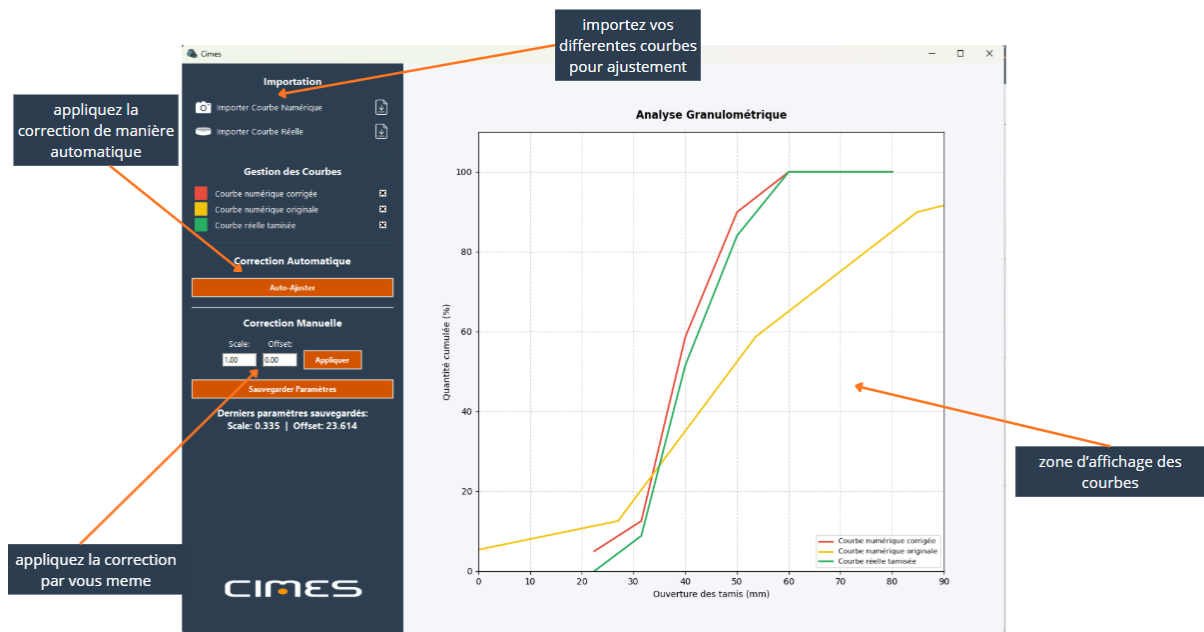
4- Interface « Recharge » du logiciel



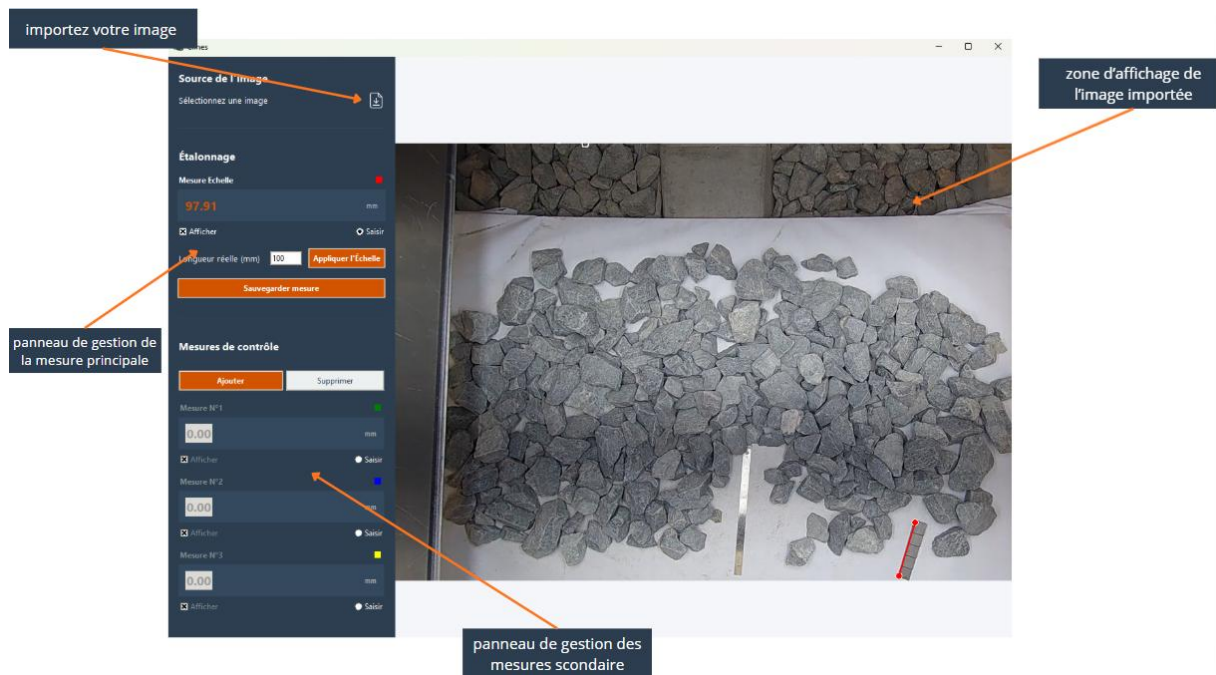
5- Interface « Paramètre » du logiciel



6- Interface « Paramètre » du logiciel



7- Interface du sous-module « Reglage-parapetre » du logiciel



8- Interface du sous-module « App-calibration » du logiciel